Computational Validation of Structural Confinement

Quantum–HPC Prototype Program for Tensor-Network and Quantum-Error-Correcting Realizations

Jeet Mozumdar

2 November 2026
Version 1.0

Abstract

This project develops two independent, minimal but scientifically meaningful computational prototypes designed to validate selected claims of the third preprint through two structurally distinct realizations of its abstract confinement framework.

Prototype A realizes the framework through finite tensor-network representability, using a controlled matrix-product-state/matrix-product- operator (MPS/MPO) setting. Prototype B realizes the framework through finite logical representability in a quantum-error-correcting-code (QEC) setting, using a stabilizer/surface-code realization.

The two prototypes are deliberately distinct in scientific objective, computational architecture, algorithmic structure, and implementation. The project does not attempt to implement the complete general theory of the preprint, nor does it attempt to prove the preprint's mathematical theorems numerically. Instead, it constructs experimentally falsifiable numerical realizations of selected assumptions and consequences of the theory.

The central objective is to establish that finite representability produces a measurable structural obstruction in two substantially different physical settings. The tensor-network prototype studies the growth of a structural observable with representational complexity. The QEC prototype studies the growth of a logical representability defect with logical protection scale.

Both prototypes are designed from the beginning as Quantum–HPC experiments: the scientific logic remains Python-accessible, while computationally intensive tensor contractions, operator transformations, stabilizer operations, and parameter sweeps can be delegated to compiled C++ kernels and parallel execution. Hybrid quantum–classical execution is supported where useful, but is not allowed to become a dependency that threatens completion of the core scientific experiments.

The target implementation period is approximately six weeks (1.5 months) for a single researcher/resource.

Contents

1 Project Definition **5**
1.1 1. Scientific Motivation and Objectives 5
1.1.1 Scientific motivation 5
1.1.2 Central research question 6
1.1.3 Project objectives 6
1.1.4 Novelty objectives 6
1.1.5 Mathematical objectives 7
1.1.6 Physical objectives 7
1.1.7 Computational objectives 8
1.1.8 Quantum–HPC objective 8
1.1.9 Quantum–HPC execution model 8

1.1.10 Optional GPU-acceleration path . . . 9
1.1.11 Hybrid quantum–classical execution . . . 10
1.1.12 Open-source software strategy . . . 10
1.1.13 Definition of computational validation . . . 11
1.2 2. Theoretical Architecture . . . 11
1.2.1 Abstract confinement architecture . . . 11
1.2.2 Prototype A theoretical architecture . . . 12
1.2.3 Prototype B theoretical architecture . . . 13
1.2.4 Commonality and independence . . . 13
1.2.5 Why the Two Prototypes Are Scientifically Distinct . . . 14
1.2.6 Obstruction mechanism . . . 14
1.2.7 Computationally testable predictions . . . 15
1.3 3. Mathematical–Computational Specification . . . 15
1.3.1 3.1 Specification principles . . . 15
1.3.2 3.2 Common mathematical objects . . . 16
1.3.3 3.3 Prototype A MCS . . . 17
1.3.4 3.4 Prototype B MCS . . . 19
1.3.5 3.5 Numerical approximation hierarchy . . . 21
1.3.6 3.6 Validation requirements . . . 21
1.3.7 3.7 Theory–computation traceability . . . 23
1.3.8 MCS freeze criteria . . . 23

2 Project Constraints **24**
2.1 Resource constraint . . . 24
2.2 Scope exclusions . . . 25
2.2.1 Prototype A exclusions . . . 25
2.2.2 Prototype B exclusions . . . 25
2.2.3 Common exclusions . . . 25

3 Master Computational Specification **26**
3.1 Computational decision hierarchy . . . 26
3.2 Common experimental protocol . . . 27
3.2.1 Physical systems . . . 27
3.2.2 Dynamics . . . 27
3.2.3 System-size scaling . . . 27
3.2.4 Representability scales . . . 28
3.2.5 Initial states . . . 28
3.2.6 Numerical tolerances . . . 28
3.3 Common software architecture . . . 29
3.4 Definition of the representability scale . . . 29
3.5 Common data structures . . . 31
3.6 TN prototype . . . 32
3.6.1 Physical model . . . 32
3.6.2 State representation . . . 33
3.6.3 TN representability subsystem . . . 33
3.6.4 TN dual map . . . 34
3.6.5 TN structural observable . . . 34
3.6.6 TN assumption test . . . 34
3.6.7 TN confinement test . . . 34
3.7 QEC prototype . . . 35
3.7.1 Code family . . . 35
3.7.2 Logical states . . . 35

3.7.3 Logical operators . . . 35
3.7.4 QEC representability scale . . . 36
3.7.5 QEC representability projection . . . 36
3.7.6 Logical representability defect . . . 36
3.7.7 Logical protection coordinate . . . 37
3.7.8 Logical representability growth test . . . 37
3.7.9 QEC observable-level A6 test . . . 37
3.7.10 QEC representable logical scale . . . 37
3.7.11 QEC confinement test . . . 38
3.8 Common algorithms . . . 38
3.8.1 Algorithm: experiment initialization . . . 38
3.8.2 Algorithm: observable evaluation . . . 39
3.8.3 Algorithm: assumption testing . . . 39
3.8.4 Algorithm: confinement testing . . . 39
3.9 Numerical decision procedures . . . 40
3.9.1 Construction of the logical growth function . . . 40
3.9.2 Sampling strategy . . . 40
3.9.3 Optimization tolerances . . . 41
3.9.4 Uncertainty aggregation . . . 41
3.10 No-circularity rule for threshold construction . . . 42
3.11 Experiment matrix . . . 43
3.12 Validation and reproducibility tests . . . 43
3.12.1 State validation . . . 43
3.12.2 Representability-map validation . . . 43
3.12.3 Convergence tests . . . 44
3.12.4 Exact small-system benchmark . . . 44
3.12.5 Reproducibility tests . . . 45
3.13 Acceptance criteria . . . 45
3.13.1 Implementation acceptance . . . 45
3.13.2 Assumption acceptance . . . 46
3.13.3 Confinement acceptance . . . 46
3.14 Failure taxonomy . . . 46
3.14.1 Failure precedence . . . 47
3.15 Output specification . . . 48
3.15.1 Raw computational data . . . 48
3.15.2 Derived observables . . . 48
3.15.3 Validation reports . . . 48
3.15.4 Assumption-status tables . . . 49
3.15.5 Confinement results . . . 49
3.16 Reproducibility package . . . 49
3.17 Final scientific deliverables . . . 50
3.18 Interpretation of computational outcomes . . . 51
3.19 Claims not established by the project . . . 51
3.20 Summary of the computational design . . . 52

4 Repository Architecture **52**
4.1 Scope of the shared framework repository . . . 53
4.2 Repository independence and future expansion . . . 53

5 Consistency and Traceability Audit of the Computational Specification 53
5.1 Realization hierarchy . . . 54
5.2 Audit status conventions . . . 55
5.3 Traceability of the six computational-design blocks . . . 55
5.4 Traceability of the thirteen blocking design decisions . . . 55
5.5 Theory–assumption–test traceability . . . 55
5.6 Hypothesis-testing traceability . . . 58
5.7 Final implementation-freeze criterion . . . 59

6 Potential Follow-Up Technical Papers 59
6.1 TN paper . . . 60
6.2 QEC paper . . . 60

A Six-Week Implementation Plan 60
A.1 Week 1: Freeze scientific definitions . . . 60
A.2 Week 2: Minimal computational cores . . . 61
A.3 Week 3: Validation infrastructure . . . 62
A.4 Week 4: Scaling experiments . . . 62
A.5 Week 5: HPC optimization and final experiments . . . 62
A.6 Week 6: Scientific consolidation . . . 63

1 Project Definition

The project is a minimal but technically rigorous computational program designed to construct two independent numerical realizations of the structural confinement mechanism developed in the third preprint. The purpose is not to reproduce the entire theoretical framework computationally, but to identify a small number of mathematically well-defined, physically meaningful, and computationally testable realizations of its central mechanism.

The project is organized according to the hierarchy

theory → mathematical–computational specification → implementation → experiment → validation → claim

The corresponding computational program consists of two scientifically independent prototypes:

A. Prototype A: Tensor-network representability confinement, implemented using an MPS/MPO realization; and

B. Prototype B: Logical representability confinement, implemented using a stabilizer/surface-code realization.

The two prototypes share the same abstract structural logic but do not share their physical representation, core numerical algorithms, or domain-specific scientific implementation.

1.1 1. Scientific Motivation and Objectives

1.1.1 Scientific motivation

The central scientific question is whether a finite representability restriction can produce a measurable structural obstruction when the representability scale is held finite while a suitable structural complexity or protection coordinate is increased.

The general conceptual chain is

finite representability ⟶ representability defect ⟶ structural lower bound ⟶ finite reachable region

The computational project asks whether this mechanism can be realized explicitly and quantitatively in two physically distinct settings.

The principal objective is therefore not merely to demonstrate that a numerical approximation becomes inaccurate at large system size. Rather, the project seeks to construct a representation-theoretically meaningful observable whose growth is tied to a structural coordinate and whose finite representability scale induces a measurable obstruction.

The two realizations are chosen precisely because they probe different computational manifestations of this principle:

Prototype A: finite tensor-network representability → F_{TN} → TN obstruction,

Prototype B: finite logical representability → O_L → logical obstruction.

The scientific value of the comparison lies in the fact that the underlying physical and computational objects are substantially different. Agreement with the same abstract confinement logic in both settings therefore provides evidence for the portability of the structural mechanism without implying that the two numerical models are equivalent.

1.1.2 Central research question

The central research question is:

> Can finite representability be implemented as an explicit computational restriction whose associated structural defect exhibits controlled growth with a structural complexity or protection coordinate, thereby producing a finite numerically reachable region consistent with the confinement mechanism of the preprint?

This question decomposes into the following computational questions:

1. Can the relevant finite representability structure be constructed explicitly?
2. Can its effect on the relevant observable be computed reproducibly?
3. Does the observable exhibit systematic dependence on structural complexity or logical protection?
4. Does this dependence persist under numerical convergence and control tests?
5. Does the extracted finite threshold agree with the scaling behavior predicted by the theoretical framework?

1.1.3 Project objectives

The project has seven primary objectives.

1. Construct a mathematically explicit finite representability mechanism for each prototype.
2. Convert that mechanism into a computable structural observable.
3. Establish a controlled scaling family in which the relevant structural coordinate can be varied independently of the representability scale.
4. Determine an empirical finite representability boundary.
5. Demonstrate that the observed boundary is not an artifact of numerical truncation, finite precision, incomplete optimization, or incorrect logical operator construction.
6. Demonstrate meaningful computational acceleration through compiled numerical kernels and parallel parameter sweeps.
7. Map every flagship numerical result to a precisely identified theoretical statement in the preprint.

The project therefore combines mathematical specification, physical realization, numerical experimentation, computational performance, and claim-level validation.

1.1.4 Novelty objectives

The project is designed to produce computational novelty beyond simply evaluating equations from the preprint.

For Prototype A, the central computational object is the relation

$$\boxed{\text{tensor-network structural complexity} \longrightarrow \text{finite-representability defect}}$$

rather than ordinary tensor-network approximation error alone.

For Prototype B, the central computational object is

$$\boxed{\text{logical protection} \longrightarrow \text{logical representability defect } O_L}$$

rather than the conventional logical error rate or decoder threshold.

The project therefore aims to establish two domain-specific diagnostics,

$$F_{\mathrm{TN}} \qquad \text{and} \qquad O_L,$$

whose scientific interpretation is explicitly tied to finite representability.

The novelty claim is deliberately narrower than a claim of a new universal theorem. The project seeks a new computational realization and diagnostic of the structural mechanism rather than a numerical proof of the general theory.

1.1.5 Mathematical objectives

The mathematical objectives are:

1. specify the state, operator, and structural spaces used by each prototype;
2. define the finite representability structure without conflating a nonlinear representable class with a linear operator subsystem;
3. establish the precise domain and codomain of every representability map;
4. reduce the defining optimization of each observable to a finite computational problem where possible;
5. identify all assumptions required for projection, closure, positivity, or complete positivity;
6. distinguish exact mathematical quantities from numerical approximations;
7. formulate the numerical test of each theoretical inequality explicitly.

No mathematical property is assumed merely because a numerical routine can be implemented. In particular, a numerical truncation routine is not identified with a mathematical projection unless the required properties have been established.

1.1.6 Physical objectives

The physical objectives are:

1. realize finite representability in a physically interpretable tensor-network setting;
2. realize finite logical representability in a quantum-error-correction setting;
3. vary structural complexity or protection independently of the finite representability scale;
4. identify the regime in which the finite representation can no longer accommodate the chosen structural family;
5. distinguish the resulting structural obstruction from conventional numerical or physical error measures.

The project therefore treats representability as a structural resource rather than merely as an implementation parameter.

1.1.7 Computational objectives

The computational implementation must satisfy four requirements:

1. **correctness:** the code implements the specified mathematical objects;
2. **reproducibility:** experiments can be regenerated from versioned configurations and recorded metadata;
3. **scalability:** the principal parameter sweeps can be executed beyond trivial single-instance calculations;
4. **traceability:** every reported scientific quantity can be traced back to a mathematical definition and forward to a validation test.

C++ is introduced for profiled computational kernels whose execution cost justifies compiled implementation. Python is the primary orchestration and analysis language.

1.1.8 Quantum–HPC objective

The HPC component is a scientific-computational demonstration rather than a separate software-development objective.

The intended pattern is

quantum-information structure + compiled numerical kernels + parallel parameter sweeps + structural validat

At least one nontrivial parallel workload must be demonstrated for each prototype.

The HPC implementation must not alter the mathematical definition of the observable. It is an implementation-level acceleration of a previously validated scientific calculation.

1.1.9 Quantum–HPC execution model

Both prototypes are executed as a layered pipeline in which Python retains control of experiment logic while computationally expensive operations are delegated to compiled C++ kernels.

Prototype A execution model. The intended execution hierarchy is

experiment manager → Python TN layer → compiled kernels → parallel CPU/HPC execution.

The likely C++ kernel candidates are:

- tensor contraction kernels;
- repeated MPO/MPS contractions;
- batched truncation/compression;
- parallel tensor operations.

Potential parallelization levels include:

1. independent system sizes;
2. independent representability scales;
3. independent structural parameters;
4. independent contraction tasks;
5. batched tensor operations.

Prototype B execution model. The execution hierarchy is

experiment manager → Python QEC layer → compiled stabilizer/logical kernels → parallel HPC execution.

The likely C++ kernel candidates are:

- stabilizer tableau operations;
- binary symplectic transformations;
- logical-operator searches;
- repeated operator-sector calculations;
- batched code-instance generation.

The workload can be distributed over:

1. code distances;
2. representability scales;
3. logical sectors;
4. noise/trajectory realizations if stochastic sampling is used;
5. repeated validation instances.

Parallel execution ordering. MPI/OpenMP-style parallel execution may be introduced only after the single-node implementation is correct and validated. Parallelization must not precede correctness.

1.1.10 Optional GPU-acceleration path

GPU acceleration is an optional extension of the compiled-kernel layer of Prototype A. It is not part of the baseline. The tensor-contraction kernels of Prototype A (repeated MPO/MPS contractions and batched truncation/compression) are its natural target: NVIDIA's cuTensorNet library, part of the cuQuantum SDK, provides GPU-accelerated tensor-network contraction, and the CUDA-Q platform builds on cuQuantum for GPU-accelerated simulation.

Licensing status. The cuTensorNet library and the cuQuantum SDK that contains it are distributed under an NVIDIA proprietary license and are not open-source software. The cuQuantum Python interface is open source (BSD-3-Clause), but it is a layer over the proprietary compiled libraries and cannot run without them, so its open-source license does not remove the exception. CUDA-Q is itself open source, but its GPU-accelerated simulation relies on cuQuantum. Use of this path is therefore a deliberate, scoped exception to the free/open-source constraint of Section 1.1.12, not a silent relaxation of it. Access to a suitable GPU, whether local or free-tier cloud, resolves the hardware question only; it does not change the licensing status of the software.

Scope of the exception. The exception is bounded by the following rules:

1. *Never required.* Every baseline result, validation test, and acceptance criterion must be reproducible entirely on the free/open-source CPU path; none may depend on the GPU path.

2. *No change of definition.* The GPU path accelerates a calculation already implemented and validated on the CPU path and must not alter the mathematical definition of any observable. Consistent with the ordering rule above, it is introduced only after the CPU implementation is correct.

3. *Cross-validation.* Wherever the CPU path is feasible, GPU-path results must agree with the corresponding CPU-path results within the declared numerical tolerance before they are reported.

4. *Explicit labeling.* Every table, figure, and configuration record produced through the GPU path is labeled as such, and the exact software versions and license terms are recorded in the reproducibility package (Section 3.16).

1.1.11 Hybrid quantum–classical execution

Hybrid execution is an optional extension rather than a critical dependency. If time permits, selected small instances may be executed using an open-source quantum simulator or a quantum backend. The purpose is not to demonstrate quantum advantage; it is to demonstrate that the structural observable and its representability analysis can form part of a hybrid workflow:

quantum state/circuit → classical structural analysis → representability observable → obstruction test.

The core results must remain reproducible entirely on free/open-source software and classical HPC resources (Section 1.1.12).

1.1.12 Open-source software strategy

The implementation relies primarily on free and open-source quantum and scientific-computing software. Candidate technologies include, subject to a final license and capability audit:

- Python/NumPy/SciPy;
- an open-source tensor-network library for the baseline path; cuTensorNet, accessed through the BSD-3-Clause-licensed cuQuantum Python interface, is considered only for the optional GPU path (Section 1.1.10), because its underlying compiled library is proprietary;
- Qiskit Aer as the named open-source quantum-circuit simulator, including its stabilizer simulation method for Clifford circuits (Section 3.12.4);
- QuEST, an open-source C/C++ state-vector and density-matrix simulator with OpenMP and MPI parallelism, whose compiled-kernel design parallels the execution model of Section 1.1.9 and which is a further candidate reference implementation (Section 3.12.4);
- Stim or an equivalent open-source stabilizer simulator;
- QuTiP, in a minor supporting role only, as an optional closed-system cross-check (Section 3.12.4);
- sparse-linear-algebra libraries;

- Eigen or equivalent C++ numerical infrastructure;
- pybind11 or equivalent C++/Python binding technology;
- OpenMP and/or MPI for HPC execution.

The final software stack must be frozen during the first implementation week. The prototypes must not depend on proprietary quantum software or paid cloud infrastructure. The single deliberate exception is the optional, clearly labeled GPU-acceleration path of Section 1.1.10, on which no baseline result depends.

1.1.13 Definition of computational validation

The computational work does not mathematically prove the general propositions of the preprint.

Instead, it tests whether selected realizations satisfy the relevant assumptions and exhibit the predicted consequences.

For each prototype, the validation chain is

$$\boxed{\text{construction} \to \text{measurement} \to \text{scaling} \to \text{threshold extraction} \to \text{theoretical comparison.}}$$

The strongest appropriate interpretation is therefore

$$\boxed{\text{numerical validation of a specified realization of the theoretical mechanism}}$$

rather than

$$\boxed{\text{numerical proof of the general theorem.}}$$

This distinction is maintained throughout the project.

1.2 2. Theoretical Architecture

1.2.1 Abstract confinement architecture

The computational architecture is derived from the structural mechanism

$$\boxed{\text{representation family} \to \text{structural objects} \to \text{finite representability subsystem} \to \text{representability map} \to \text{stru}}$$

The essential idea is that a finite representability scale defines a restricted computationally accessible structure. If a structural observable has a lower bound that grows with a structural coordinate, then sufficiently complex structures become incompatible with the finite representability restriction.

Schematically,

$$O(s, D) \geq g_D(s),$$

where s denotes the relevant structural coordinate and D denotes the finite representability scale.

If the representability constraint imposes

$$O(s, D) \leq \varepsilon_D(T),$$

then a structural obstruction follows whenever

$$g_D(s) > \varepsilon_D(T).$$

The computational objective is to determine whether this structure can be realized numerically and whether the resulting finite threshold is consistent with the predicted growth relation.

1.2.2 Prototype A theoretical architecture

Prototype A realizes the mechanism using matrix-product states and matrix-product operators.

The intended computational chain is

$$\boxed{\text{MPS/MPO family} \to \text{finite TN representability} \to \Pi_D^{(\mathrm{TN})} \to F_{\mathrm{TN}} \to d_{\mathrm{TN}} \to d_{\max}^{(\mathrm{TN})}(D) \to \text{obstruction.}}$$

A representative state is

$$|\psi\rangle \in \mathcal{H}_N, \qquad \mathcal{H}_N = (\mathbb{C}^q)^{\otimes N},$$

with an MPS representation characterized by bond dimensions and associated structural quantities.

Operators are represented by MPOs. A finite representability scale is introduced through an explicitly specified restriction on the admissible MPO representation.

A critical mathematical distinction is imposed here:

$$\boxed{\text{bounded-bond-dimension MPO class} \neq \text{automatically a linear operator subsystem.}}$$

The bounded-bond-dimension class is generally nonlinear under addition and therefore cannot automatically be treated as an operator system admitting a linear projection.

Accordingly, the MCS must distinguish between

$$\mathfrak{R}_D^{(\mathrm{TN})} = \{\text{operators representable at scale } D\}$$

and, where required by the theoretical framework, a generated linear/operator subsystem

$$\mathcal{A}_D^{(\mathrm{TN})} = \mathrm{Gen}\Big(\mathfrak{R}_D^{(\mathrm{TN})}\Big).$$

The precise choice of generated structure must be justified rather than assumed.

Only after this distinction is resolved may a map

$$\Pi_D^{(\mathrm{TN})}$$

be identified with a mathematical projection. If the computational procedure is instead a nonlinear approximation or compression map, it must be denoted and analyzed accordingly.

The structural observable is represented schematically by

$$F_{\mathrm{TN}}(\rho) = \sup_{\substack{X \in \mathcal{A}_{\mathrm{TN}} \\ \|X\| \leq 1}} \left| \mathrm{Tr}(X\rho) - \mathrm{Tr}\left[\Pi_D^{(\mathrm{TN})}(X)\rho\right] \right|,$$

subject to the precise operator-space and projection definitions fixed by the MCS.

The structural coordinate d_{TN} is not assumed a priori to be identical to MPS bond dimension. Bond dimension, entanglement measures, operator complexity, support, and other candidate coordinates must be distinguished and the coordinate used in the final scaling law must be explicitly justified.

1.2.3 Prototype B theoretical architecture

Prototype B realizes the mechanism using a stabilizer-code, with a surface-code family as the preferred implementation.

The intended computational chain is

$$\boxed{\text{stabilizer code} \to \text{logical-operator extraction} \to \mathcal{A}_D^{(\mathrm{QEC})} \to \Pi_D^{(\mathrm{QEC})} \to O_L \to d_{\max}^{(\mathrm{QEC})}(D) \to \text{obstruction.}}$$

For a stabilizer group $\mathcal{S}$, the logical Pauli structure is associated with

$$N(\mathcal{S})/\mathcal{S}.$$

The implementation must computationally determine physical representatives of the nontrivial logical sectors rather than rely exclusively on hard-coded logical-operator formulas.

The resulting data must retain at least

$$\{\text{logical sector}, \text{representative}, \text{support}, \text{weight}, \text{commutation relations}, \text{protection scale}\}.$$

A finite logical representability structure is then constructed,

$$\mathcal{A}_D^{(\mathrm{QEC})},$$

from explicitly defined logically accessible operators.

The construction must specify whether this object is a set, span, operator system, algebra, or other mathematical structure.

The corresponding representability operation

$$\Pi_D^{(\mathrm{QEC})}$$

must likewise be defined mathematically rather than inferred from a numerical filtering procedure.

The logical representability observable is schematically

$$O_L(\rho) = \sup_{\substack{X \in \mathcal{A}_{\log} \\ \|X\| \le 1}} \left| \mathrm{Tr}(X\rho) - \mathrm{Tr}\left[\Pi_D^{(\mathrm{QEC})}(X)\rho\right] \right|.$$

The exact finite-dimensional reduction of this optimization is a central MCS task.

The logical protection coordinate must also be defined carefully. In particular, the following quantities must not be identified without justification:

$$d_{\mathrm{code}}, \qquad \mathrm{wt}(L), \qquad d_L.$$

The final definition of d_L must correspond to the structural coordinate appearing in the theoretical growth condition.

1.2.4 Commonality and independence

The two prototypes share a common abstract vocabulary:

StructuralState, StructuralObservable, RepresentabilityStructure, RepresentabilityMap, Expe

However,

common mathematical vocabulary $\neq$ common scientific implementation.

The TN and QEC repositories remain independent.

Their commonality exists at the level of experiment management, metadata, validation philosophy, numerical analysis, and abstract structural concepts. Their physical realizations, algorithms, and domain-specific data structures remain separate.

1.2.5 Why the Two Prototypes Are Scientifically Distinct

The two prototypes deliberately operate on different structural objects.

	Prototype A: TN	**Prototype B: QEC**
Physical setting	Tensor-network representation	Encoded quantum information
Primary object	MPS/MPO representability	Logical operator structure
Structural coordinate	Tensor-network structural complexity	Logical protection scale d_L
Representability mechanism	Finite tensor/operator representation	Finite logical operator subsystem
Observable	Tensor-network representability defect	Logical defect O_L
Main computational operation	Tensor contractions, truncations, MPO action	Stabilizer/logical-operator manipulation
Primary scaling variable	Bond/representation scale and structural complexity	Code size/logical operator support
Main obstruction	Structures requiring representation beyond finite TN scale	Structures requiring logical information beyond finite logical scale
Quantum–HPC character	Large tensor contractions and parameter sweeps	Large stabilizer/operator simulations and ensemble sweeps

Thus the computational similarity is deliberately limited to the abstract logic of the preprint. The numerical realization is different.

1.2.6 Obstruction mechanism

For either prototype, the computational obstruction is defined by the intersection of two conditions.

First, the theoretical structural growth condition:

$$O(s, D) \gtrsim g_D(s).$$

Second, the finite representability constraint:

$$O(s, D) \leq \varepsilon_D(T).$$

The predicted boundary is therefore determined implicitly by

$$g_D(s_{\max}) \sim \varepsilon_D(T).$$

Numerically, the corresponding quantity is denoted

$$d_{\max}^{\mathrm{num}}(D).$$

The central experiment is therefore not simply to observe that an error grows. It is to determine whether a reproducible finite boundary emerges whose dependence on D is consistent with the theoretical growth mechanism.

1.2.7 Computationally testable predictions

The project tests the following classes of predictions.

1. The representability defect can be defined and evaluated at finite representability scale.
2. The defect is systematically dependent on the selected structural coordinate.
3. Increasing representability scale changes the location of the empirical obstruction boundary.
4. The scaling persists under numerical convergence tests.
5. The measured growth is distinguishable from ordinary numerical approximation error.
6. The resulting finite boundary is consistent with the relevant theoretical growth condition over the computationally accessible regime.

These are computational hypotheses rather than universal mathematical claims.

1.3 3. Mathematical–Computational Specification

The Mathematical–Computational Specification (MCS) is the formal bridge between the theoretical architecture and the implementation.

Its purpose is to ensure that every scientific quantity used by the project has an explicit mathematical definition, computational representation, algorithm, approximation hierarchy, validation procedure, and traceability path.

The governing principle is

mathematical object → computational representation → algorithm → numerical approximation → validation

The MCS is therefore treated as the scientific contract for the implementation. No downstream implementation should introduce a scientific object that has not been specified here or explicitly added through a controlled revision.

1.3.1 3.1 Specification principles

The MCS obeys the following principles.

Principle 1: Mathematical primacy. The computational implementation must represent a previously specified mathematical object. Code structure must not determine the scientific definition.

Principle 2: Explicit domains and codomains. Every map must have a specified domain and codomain. In particular,

$$\Pi_D : \mathcal{X} \to \mathcal{Y}$$

must not be described simply as a "projection" without specifying $\mathcal{X}$, $\mathcal{Y}$, and the properties required of the map.

Principle 3: Structural distinctions must be preserved. The following distinctions must not be collapsed without proof or explicit justification:

$$\begin{aligned}&\text{representable set} \neq \text{linear span},\\ &\text{approximation map} \neq \text{mathematical projection},\\ &\text{bond dimension} \neq \text{structural complexity},\\ &\text{code distance} \neq \text{logical-operator complexity},\\ &\text{numerical error} \neq \text{representability defect}.\end{aligned}$$

Principle 4: Finite computational reduction. Every formally defined supremum, infimum, optimization, or operator norm must either be evaluated exactly for the finite instance or reduced to a documented finite-dimensional numerical problem.

Principle 5: Approximation transparency. All approximations must be separately identified. In particular,

$$\text{physical approximation}, \quad \text{representability approximation}, \quad \text{finite-size effect}, \quad \text{optimization error},$$

must not be combined into an unexplained single error quantity.

Principle 6: Independent validation. Selected observables must be independently recomputable by an alternative algorithm, exact small-system calculation, analytical benchmark, or symmetry constraint.

Principle 7: No silent assumptions. If a mathematical property required by the theory has not been established, the MCS records it as an explicit unresolved item rather than assuming it from the existence of a numerical routine.

Principle 8: Reproducibility. Every numerical result must be associated with:

$$\{\text{code version}, \text{configuration}, \text{parameters}, \text{random seed}, \text{numerical tolerances}, \text{hardware/software environment}\}.$$

1.3.2 3.2 Common mathematical objects

The two prototypes are represented through a common abstract structure.

Let

$$\mathcal{X}$$

denote the relevant physical or logical state/operator space, $\mathfrak{R}_D$ the finite-scale representability structure, and Π_D the corresponding representability operation when such an operation is mathematically justified.

A generic structural observable has the form

$$O_D(\rho) = \sup_{\substack{X \in \mathcal{A} \\ \|X\| \leq 1}} \left| \mathrm{Tr}\left[X\rho\right] - \mathrm{Tr}\left[\Pi_D(X)\rho\right] \right|.$$

The implementation must specify:

$$\boxed{(\mathcal{X}, \mathcal{A}, \mathfrak{R}_D, \Pi_D, O_D)}$$

for each realization.

The common computational metadata associated with each observable are

$$\mathcal{M} = \{D,\ s,\ N,\ O_D,\ \delta_{\text{num}},\ \delta_{\text{rep}},\ \delta_{\text{fs}}\},$$

where s is the structural coordinate and the remaining quantities record relevant representability, numerical, and finite-size information.

The common experiment abstraction is

$$\mathcal{E} : \Theta \longrightarrow \mathcal{D},$$

where Θ is a versioned parameter specification and $\mathcal{D}$ is the resulting structured dataset.

The common validation abstraction is

$$\mathcal{V} : \mathcal{D} \longrightarrow \{\text{pass}, \text{fail}, \text{qualified}\}.$$

No result is treated as a scientific output until the corresponding validation criteria have been satisfied.

1.3.3 3.3 Prototype A MCS

3.3.1 State space. For a chain of N sites with local Hilbert-space dimension q,

$$\mathcal{H}_N = (\mathbb{C}^q)^{\otimes N}.$$

The prototype initially uses one-dimensional tensor-network states represented as MPS:

$$|\psi\rangle = \sum_{\{s_i\}} A_1^{s_1} A_2^{s_2} \cdots A_N^{s_N} |s_1 s_2 \cdots s_N\rangle.$$

The implementation must support canonicalization, normalization, bond-dimension control, and extraction of appropriate structural diagnostics.

3.3.2 Operator representation. Operators are represented as MPOs,

$$X = \sum_{\{s_i, t_i\}} W_1^{s_1 t_1} W_2^{s_2 t_2} \cdots W_N^{s_N t_N} |s_1 \cdots s_N\rangle\langle t_1 \cdots t_N|.$$

The implementation must support MPO construction, contraction, compression, and expectation-value evaluation.

3.3.3 Finite representability. A candidate finite representability class is

$$\mathfrak{R}_D^{(\text{TN})} = \{X : X \text{ admits an MPO representation with bond dimension} \leq D\}.$$

This definition is computationally natural but is not, by itself, sufficient to establish that $\mathfrak{R}_D^{(\text{TN})}$ is a linear operator subsystem.

Therefore the MCS requires an explicit decision between the following possibilities:

$$\begin{cases} \text{use } \mathfrak{R}_D^{(\text{TN})} \text{ directly as a nonlinear representability class,} \\ \text{construct a generated linear/operator subsystem } \mathcal{A}_D^{(\text{TN})}, \\ \text{or define another mathematically justified finite-scale structure.} \end{cases}$$

The choice must be compatible with the assumptions of the theoretical framework.

3.3.4 Representability operation. A numerical MPO compression routine may produce a map

$$C_D : X \mapsto X_D,$$

but C_D is not automatically a projection.

The implementation must therefore distinguish:

$$C_D \quad \text{(numerical compression/approximation)}$$

from

$$\Pi_D^{(\mathrm{TN})} \quad \text{(mathematical representability projection)},$$

if the latter is required by the theory.

If a true projection is required, the MCS must verify the relevant properties, such as

$$\Pi_D^2 = \Pi_D,$$

and any required linearity, positivity, complete positivity, norm contraction, or adjointness properties.

3.3.5 Structural coordinate. The structural coordinate is denoted

$$d_{\mathrm{TN}}.$$

It must be defined independently of implementation convenience.

Candidate quantities may include bond dimension, operator Schmidt rank, entanglement-related quantities, support complexity, or another structural quantity dictated by the preprint.

The final coordinate must satisfy the requirement that the theoretical growth condition is formulated in terms of the same quantity that is measured numerically.

3.3.6 Structural observable. The TN observable is denoted

$$F_{\mathrm{TN}}(\rho) = \sup_{\substack{X \in \mathcal{A}_{\mathrm{TN}} \\ \|X\| \le 1}} \left| \mathrm{Tr}(X\rho) - \mathrm{Tr}\left[\Pi_D^{(\mathrm{TN})}(X)\rho\right] \right|.$$

The exact domain $\mathcal{A}_{\mathrm{TN}}$, representability structure, map $\Pi_D^{(\mathrm{TN})}$, and norm must be fixed before production experiments.

Only one observable is treated as the flagship structural witness for Prototype A. Any additional observable computed for cross-checking purposes is explicitly designated secondary and is not used, on its own, to support or refute the confinement claim.

3.3.7 Numerical reduction. The continuous supremum must be reduced to a finite computational problem.

Possible reductions include a finite operator basis, a matrix norm, a convex optimization problem, or an equivalent finite-dimensional dual formulation.

The selected reduction must be mathematically justified and benchmarked against exact calculations for small systems.

3.3.8 Error hierarchy. Prototype A must separately track

$$\delta_{\mathrm{TN}}^{\mathrm{rep}}, \qquad \delta_{\mathrm{TN}}^{\mathrm{trunc}}, \qquad \delta_{\mathrm{TN}}^{\mathrm{fs}}, \qquad \delta_{\mathrm{TN}}^{\mathrm{opt}}, \qquad \delta_{\mathrm{TN}}^{\mathrm{fp}},$$

corresponding, where applicable, to representability, tensor truncation, finite-size, optimization, and floating-point effects.

The central scientific requirement is

$$\boxed{F_{\mathrm{TN}} \neq \text{uncontrolled tensor-network numerical error.}}$$

3.3.9 Primary numerical prediction. For fixed D, the principal experiment seeks evidence for a relation of the form

$$F_{\mathrm{TN}}(d_{\mathrm{TN}}, D) \gtrsim g_D(d_{\mathrm{TN}})$$

over the numerically accessible regime.

The empirical obstruction threshold is defined operationally by the crossing

$$F_{\mathrm{TN}}(d_{\mathrm{TN}}, D) > \varepsilon_D(T),$$

or by the corresponding criterion required by the final theoretical formulation.

This produces

$$d_{\max}^{\mathrm{TN,num}}(D).$$

1.3.4 3.4 Prototype B MCS

3.4.1 Code family. The preferred initial realization is a surface/stabilizer code family with increasing code size and logical protection.

For stabilizer generators

$$\mathcal{S} = \langle S_1, \ldots, S_r \rangle,$$

the implementation represents Pauli operators using a binary symplectic representation.

3.4.2 Logical operator space. Logical operators are computationally extracted from

$$N(\mathcal{S})/\mathcal{S}.$$

The implementation must identify independent logical sectors and corresponding physical representatives.

For each representative L, the implementation records

$$\{L,\ \mathrm{supp}(L),\ \mathrm{wt}(L),\ \text{logical sector, commutation data}\}.$$

3.4.3 Logical structural coordinate. The logical protection coordinate is denoted

$$d_L.$$

The implementation must distinguish this quantity from the raw physical code distance whenever the two are not mathematically identical.

The final definition is required to match the structural coordinate appearing in the theoretical growth condition.

3.4.4 Finite logical representability subsystem. A finite logical structure is denoted

$$\mathcal{A}_D^{(\mathrm{QEC})}.$$

It must be generated from a clearly specified accessibility criterion.

The implementation must determine whether the resulting object is

$$\text{a set}, \qquad \text{linear span}, \qquad \text{operator system}, \qquad \text{algebra},$$

or another structure.

It must not be defined merely as "all logical operators of weight below D" unless that construction is demonstrably compatible with the required mathematical structure.

3.4.5 Representability map. The map

$$\Pi_D^{(\mathrm{QEC})}$$

must be explicitly defined on the chosen logical operator space.

If it is a projection, the implementation must verify the relevant properties. If it is instead a finite-scale approximation or restriction map, its mathematical status must be stated accordingly.

3.4.6 Logical representability observable. The principal observable is

$$O_L(\rho) = \sup_{\substack{X \in \mathcal{A}_{\log} \\ \|X\| \le 1}} \left| \mathrm{Tr}(X\rho) - \mathrm{Tr}\left[\Pi_D^{(\mathrm{QEC})}(X)\rho\right] \right|.$$

The computational implementation must reduce this quantity to a finite optimization or equivalent norm calculation.

3.4.7 Logical-error distinction. The quantity O_L is explicitly distinguished from conventional QEC diagnostics:

$$O_L \neq p_{\mathrm{logical}},$$

and

$$O_L \neq \text{decoder failure probability},$$

and

$$O_L \neq \text{syndrome error rate}.$$

The scientific interpretation of O_L concerns logical operator representability rather than ordinary stochastic logical failure.

3.4.8 Primary numerical prediction. The principal experiment seeks evidence for

$$O_L(d_L, D) \gtrsim g_D(d_L)$$

over the numerically accessible regime.

The empirical threshold is obtained from the relevant representability criterion, for example

$$O_L(d_L, D) > \varepsilon_D(T),$$

yielding

$$d_{\max}^{\mathrm{QEC,num}}(D).$$

3.4.9 Error and validation hierarchy. Prototype B must separately control:

$$\delta_{\text{logical}}, \qquad \delta_{\text{rep}}, \qquad \delta_{\text{opt}}, \qquad \delta_{\text{fs}}, \qquad \delta_{\text{fp}},$$

representing logical-operator construction error, representability error, optimization error, finite-size effects, and floating-point effects as applicable.

The central requirement is

$$\boxed{O_L \neq \text{ordinary logical error rate.}}$$

1.3.5 3.5 Numerical approximation hierarchy

Every reported quantity must be understood through the hierarchy

$$\boxed{\text{exact mathematical quantity} \rightarrow \text{finite-dimensional formulation} \rightarrow \text{numerical algorithm} \rightarrow \text{finite-precision resu}}$$

The project distinguishes at least five error classes.

1. **Representation error:** error induced by replacing the full structural space with a finite representability structure.
2. **Physical/state approximation error:** error arising from an approximate representation of the underlying physical state or code state.
3. **Finite-size error:** dependence on system size or code size.
4. **Algorithmic error:** truncation, incomplete optimization, iterative convergence, or approximation introduced by the numerical algorithm.
5. **Floating-point error:** finite numerical precision and accumulation of roundoff.

The observable of interest must not be identified with any one of these numerical errors.

Convergence studies therefore vary relevant control parameters independently, for example

$$\chi_{\max}, \qquad D, \qquad N, \qquad \epsilon_{\text{trunc}}, \qquad \epsilon_{\text{opt}},$$

for Prototype A, and

$$d_{\text{code}}, \qquad D, \qquad \epsilon_{\text{opt}}, \qquad \epsilon_{\text{fp}},$$

for Prototype B.

A production result is accepted only when the reported scientific trend is stable under the appropriate numerical controls.

1.3.6 3.6 Validation requirements

Validation is organized into four levels.

Level 1: implementation validation. The implementation must reproduce known mathematical properties.

For Prototype A:

$$\langle\psi|\psi\rangle = 1,$$

correct tensor dimensions, correct MPO contractions, and appropriate canonical-form identities must be verified.

For Prototype B:

$$[S_i, S_j] = 0,$$

and the extracted logical operators must satisfy the required logical commutation relations.

Level 2: numerical validation. Numerical stability is tested through:

- precision variation;
- convergence with truncation tolerance;
- convergence with bond dimension;
- convergence with code/system size;
- independent recomputation;
- exact small-instance comparisons, including cross-checks against independent open-source reference implementations (Section 3.12.4);
- comparison against an intentionally over-resolved tensor-network reference (bond dimension substantially larger than every tested D, but not necessarily at the exact-diagonalization scale), used to separate the representability defect from ordinary truncation error at system sizes beyond exact-benchmark reach.

Level 3: structural validation. The measured quantity must have the intended scientific interpretation.

For Prototype A:

$$F_{\mathrm{TN}} \neq \text{ordinary tensor-network truncation artifact.}$$

For Prototype B:

$$O_L \neq \text{ordinary logical error rate.}$$

This requires control experiments designed to separate the representability effect from conventional numerical or QEC performance measures.

Level 4: theoretical validation. The final data are compared against the relevant theoretical growth condition:

$$F_{\mathrm{TN}} \gtrsim g_D(d_{\mathrm{TN}})$$

and

$$O_L \gtrsim g_D(d_L),$$

with the corresponding finite-scale obstruction criterion.

The comparison is quantitative wherever the theory provides a quantitative prediction and qualitative only where the theoretical statement is qualitative.

1.3.7 3.7 Theory–computation traceability

Every flagship computational result must admit the traceability chain

theoretical statement → mathematical object → algorithm → repository module → experiment → validation

The traceability structure for Prototype A is

$$\begin{gathered}\text{TN theoretical statement}\\ \downarrow\\ \text{finite TN representability}\\ \downarrow\\ \mathcal{A}_D^{(\mathrm{TN})},\ \Pi_D^{(\mathrm{TN})}\\ \downarrow\\ F_{\mathrm{TN}}\\ \downarrow\\ F_{\mathrm{TN}}(d_{\mathrm{TN}}, D)\\ \downarrow\\ d_{\max}^{\mathrm{TN,num}}(D)\\ \downarrow\\ \text{validated TN obstruction.}\end{gathered}$$

The corresponding chain for Prototype B is

$$\begin{gathered}\text{QEC theoretical statement}\\ \downarrow\\ N(\mathcal{S})/\mathcal{S}\\ \downarrow\\ \mathcal{A}_D^{(\mathrm{QEC})},\ \Pi_D^{(\mathrm{QEC})}\\ \downarrow\\ O_L\\ \downarrow\\ O_L(d_L, D)\\ \downarrow\\ d_{\max}^{\mathrm{QEC,num}}(D)\\ \downarrow\\ \text{validated logical obstruction.}\end{gathered}$$

The exact proposition, theorem, assumption, or equation numbers from the final version of the third preprint must be inserted into the traceability matrix before production experiments begin. The design freeze recorded in this document (Section 5.7) precedes that insertion. The insertion is a separate, versioned revision of the traceability matrix and does not reopen any design decision frozen here.

No experiment is described as validating a theoretical claim unless the corresponding theoretical statement has first been explicitly identified.

1.3.8 MCS freeze criteria

Before implementation enters the production phase, the following quantities must be mathematically frozen:

representability structure,
representability scale D,
structural coordinate,
representability map,
observable,
norm/optimization domain,
obstruction criterion,
numerical tolerance definitions.

In particular, the project does not regard the expressions

$$\Pi_D^{(\mathrm{TN})}, \qquad \Pi_D^{(\mathrm{QEC})}, \qquad F_{\mathrm{TN}}, \qquad O_L$$

as fully specified merely because symbolic formulas have been written for them. Their mathematical domains, computational realizations, and numerical evaluation procedures must all be fixed.

Any unresolved item at this stage is recorded explicitly as an MCS decision item and resolved before the corresponding production experiment.

The MCS therefore functions as the formal scientific gate between the theoretical project definition and the implementation work.

Project Definition → MCS freeze → Implementation → Experiment

2 Project Constraints

The project is deliberately constrained so that the two prototypes can be completed by one researcher in approximately six weeks.

2.1 Resource constraint

The planning assumption is:

one researcher/resource

with approximately

1.5 months ≈ 6 weeks

of implementation time.

The project therefore prioritizes:

- scientific validity,
- reproducibility,
- clear numerical scaling,
- independent implementation of the two realizations,
- Quantum–HPC relevance,
- computational novelty,

- and publication-quality experimental outputs.

It does *not* prioritize maximal physical realism or exhaustive coverage of code and tensor-network families.

2.2 Scope exclusions

The following items are deliberately excluded from the six-week prototype program.

2.2.1 Prototype A exclusions

- multiple tensor-network families as primary experiments;
- PEPS as a full second implementation;
- MERA as a full implementation;
- large-scale 2D tensor-network calculations;
- exhaustive tensor-network classification;
- development of a new tensor-network library;
- proving general tensor-network complexity theorems;
- exhaustive optimization of tensor contraction paths;
- proprietary HPC infrastructure as a dependency of any baseline result (the optional GPU path of Section 1.1.10 is a labeled, non-required exception).

2.2.2 Prototype B exclusions

- simultaneous implementation of many QEC families;
- full fault-tolerant quantum-computing architecture;
- complete decoder benchmarking;
- logical-threshold estimation as the primary scientific objective;
- experimental hardware execution as a requirement;
- exhaustive stabilizer-code classification;
- development of a new QEC simulator from scratch;
- proof that all QEC families satisfy the representability growth condition;
- universal statements about quantum error correction.

2.2.3 Common exclusions

Neither prototype will attempt:

- to reproduce the entire preprint computationally;
- to implement every axiom independently;
- to establish mathematical proofs through numerical experiments;

- to build a production-grade quantum-HPC framework;
- to optimize for quantum advantage;
- to support every possible quantum software stack;
- to provide exhaustive physical parameter studies.

3 Master Computational Specification

Purpose and scope. This section specifies the computational project used to investigate the tensor-network (TN) and quantum-error-correction (QEC) realizations of the structural confinement framework developed in the preceding theoretical sections.

The computational project is not intended to repair, strengthen, or independently prove the abstract theory. In particular, no computational result is interpreted as a proof of Axioms A1–A6 or of the general confinement theorem. Instead, the project evaluates concrete realizations of the abstract framework in three logically distinct stages:

$$\boxed{\text{Implementation} \longrightarrow \text{Assumption testing} \longrightarrow \text{Confinement testing.}}$$

The computational project therefore addresses four questions:

1. Can the proposed TN and QEC representability structures be implemented faithfully and numerically validated?
2. Do the additional realization-dependent assumptions required by the two concrete constructions hold in the selected computational systems?
3. Conditional on the implementation and realization assumptions, does the predicted structural confinement appear numerically?
4. If an implementation condition or realization assumption fails, where does the proposed mechanism fail, and can that failure be quantitatively localized?

The computational project consequently distinguishes between failure of an implementation, failure of a realization assumption, failure of a predicted confinement consequence, and numerical inconclusiveness. No failure of a concrete realization is interpreted as a refutation of the abstract framework.

3.1 Computational decision hierarchy

The computational logic is organized according to the following hierarchy:

$$\boxed{\mathsf{I} \to \mathsf{A} \to \mathsf{C}}$$

where

I = implementation validation, A = realization-assumption testing, C = confinement testing.

The four possible high-level outcomes are:

$$\begin{aligned}
\text{Implementation fails} &\implies \text{the prototype cannot test the proposed mechanism,}\\
\text{Implementation passes, assumption fails} &\implies \text{the selected realization does not support the mechanism,}\\
\text{Assumptions pass, confinement fails} &\implies \text{the predicted computational consequence is not observed,}\\
\text{Assumptions pass, confinement is observed} &\implies \text{the computational data support the proposed mechanism.}
\end{aligned}$$

A fifth outcome,

$$\text{INCONCLUSIVE},$$

is assigned whenever numerical precision, finite-size effects, sampling, optimization error, or insufficient separation from the relevant threshold prevents a reliable classification.

The computational decision tree is therefore:

$$\boxed{\text{Implementation} \rightarrow \text{Assumption testing} \rightarrow \text{Confinement testing}}$$

with no transition to a later stage when a required earlier stage has failed.

3.2 Common experimental protocol

The TN and QEC prototypes share a common experimental protocol wherever the physical realizations permit such a comparison.

3.2.1 Physical systems

The TN realization uses a one-dimensional open chain of qubits,

$$\mathcal{H}_N = \bigotimes_{j=1}^{N} \mathbb{C}^2,$$

while the QEC realization uses a two-dimensional planar surface-code architecture with linear size L.

The different geometries are intentional. The common computational framework concerns the representability and reachability logic rather than requiring the two realizations to possess identical microscopic geometry.

3.2.2 Dynamics

The primary computational experiments use local, time-independent Hamiltonian evolution,

$$\rho(T) = U(T)\rho_0 U(T)^\dagger, \qquad U(T) = e^{-iHT}.$$

The primary experiments therefore avoid introducing stochastic physical noise as an additional source of uncertainty. Noise models may be introduced only as an explicitly identified extension and are not part of the baseline confinement test.

For each model, the evolution is evaluated at a fixed time grid

$$T_0, T_1, \ldots, T_{N_T},$$

with additional time points introduced near an empirically observed representability threshold when required.

3.2.3 System-size scaling

Each prototype is evaluated at a sequence of increasing system sizes.

For the TN prototype,

$$N \in \mathcal{N}_{\mathrm{TN}},$$

where the baseline implementation should contain at least four increasing system sizes whenever computationally feasible.

For the QEC prototype,

$$L \in \mathcal{L}_{\mathrm{QEC}},$$

with the baseline sequence consisting of at least four increasing code sizes whenever computationally feasible.

Finite-size extrapolation is never used as the sole basis for a confinement claim. Raw finite-size results are reported first, and scaling analyses are used only to determine whether the observed behavior is stable under system growth.

3.2.4 Representability scales

The TN representability scale is the MPO bond-dimension cutoff,

$$D \in \mathcal{D}_{\mathrm{TN}},$$

with a baseline sequence of increasing values such as

$$D \in \{2, 4, 8, 16, \ldots\},$$

subject to computational feasibility.

The QEC representability scale is the logical accessibility/support scale,

$$D \in \mathcal{D}_{\mathrm{QEC}},$$

where D specifies the maximum physical support or implementation scale admitted into the accessible logical operator subsystem.

The symbol D therefore denotes a finite representability scale in both prototypes, but its concrete physical meaning differs between them.

3.2.5 Initial states

Both prototypes use a fixed reference ensemble and a structured target ensemble.

Reference states provide baseline behavior for which the representability defect is expected to be small or otherwise controlled. Structured states are selected to probe the regime in which the relevant structural coordinate is expected to become difficult to represent.

The complete initial-state specification is stored with every computational run and includes:

$$\{\rho_0, \text{state label}, \text{system size}, D, \text{seed}, \text{Hamiltonian parameters}\}.$$

3.2.6 Numerical tolerances

A numerical tolerance

$$\tau_{\mathrm{num}} > 0$$

is introduced independently of the theoretical compatibility error

$$\varepsilon_D(T).$$

The distinction

$$\tau_{\mathrm{num}} \neq \varepsilon_D(T)$$

is mandatory.

A theoretical inequality such as

$$O(\rho(T)) \leq \varepsilon_D(T)$$

is considered numerically compatible whenever the measured violation is within the declared numerical uncertainty.

For a measured quantity Q, the computational pipeline records

$$Q_{\mathrm{meas}} \pm \delta Q_{\mathrm{num}},$$

where δQ_{num} contains the relevant truncation, roundoff, discretization, optimization, and sampling contributions.

3.3 Common software architecture

The computational project is organized into the following modules:

1. `config`: global experiment configuration and parameter validation;
2. `states`: construction, normalization, and validation of physical and encoded states;
3. `dynamics`: time evolution and evolution diagnostics;
4. `representability`: common interfaces for finite-scale representability maps;
5. `observables`: computation of structural observables and associated numerical uncertainties;
6. `assumptions`: evaluation of realization-dependent assumptions;
7. `reachability`: generation and classification of reachable structural data;
8. `confinement`: threshold calculation and forbidden-region tests;
9. `validation`: unit tests, convergence tests, exact benchmarks, and structural-map diagnostics;
10. `experiments`: experiment orchestration and parameter sweeps;
11. `analysis`: statistical analysis, scaling analysis, and classification of outcomes;
12. `outputs`: serialization of raw data, processed results, figures, tables, and experiment metadata;
13. `reproducibility`: seed management, environment capture, configuration snapshots, and provenance tracking.

The software architecture must maintain a strict separation between:

physical simulation	representability map	observable	hypothesis test.

No analysis module may silently modify the physical simulation or the definition of the representability map.

The common framework itself must contain only genuinely generic, domain-agnostic functionality (configuration, metadata, scheduling, serialization, logging, and reproducibility utilities). Any component that encodes scientific content specific to the TN or QEC realization belongs to the corresponding prototype repository, not to the common framework.

3.4 Definition of the representability scale

The symbol D denotes the finite representability scale of a computational realization. The abstract role of D is to parameterize the restricted representability structure used to construct the finite-scale map

$$\Pi_D.$$

The meaning of D is realization-dependent and is not assumed to be the same numerical quantity in different prototypes.

For each prototype, the computational implementation must specify:

$$D \longrightarrow \text{concrete realization parameter} \longrightarrow \Pi_D.$$

Table 1: Assignment of software-architecture modules (Section 3.3) to repositories (Section 4).

Module	**Repository assignment**
`config`	Framework. Global configuration schema and parameter validation are realization-agnostic.
`states`	TN and QEC, implemented independently. State construction and validation are realization-specific (MPS/MPO states for TN; encoded/stabilizer states for QEC).
`dynamics`	TN and QEC, implemented independently. The evolution model is shared in the abstract ($U(T) = e^{-iHT}$), but its computational realization differs by prototype.
`representability`	Framework provides the generic interface for finite-scale representability maps; TN and QEC each implement the concrete map (Π_D^{MPO} for TN; the logical accessibility projection for QEC).
`observables`	TN and QEC, implemented independently. F_{TN} and O_L are distinct, realization-specific observables.
`assumptions`	TN and QEC, implemented independently, since the evaluated assumptions are realization-dependent by definition.
`reachability`	TN and QEC, implemented independently. The structural data being classified differs by realization.
`confinement`	Framework provides the generic threshold-comparison and classification harness; TN and QEC each supply their own threshold function ($\varepsilon_D(T)$ for TN; $g_D(d_L)$ for QEC).
`validation`	Framework provides generic testing utilities and the audit-status vocabulary (Section 5.6); exact benchmarks, convergence tests, and structural-map diagnostics are realization-specific and live in the TN and QEC repositories.
`experiments`	Framework provides generic orchestration and parameter-sweep scheduling; concrete experiment definitions (baseline, scaling, obstruction) are realization-specific.
`analysis`	Framework. Statistical and scaling-analysis utilities are realization-agnostic.
`outputs`	Framework. Serialization, metadata, and result management are realization-agnostic.
`reproducibility`	Framework. Seed management, environment capture, and provenance tracking are realization-agnostic.

The computational project therefore distinguishes:

$$D_{\mathrm{TN}} \neq D_{\mathrm{QEC}}$$

as implementation parameters, while retaining the common notation D for their shared role as finite representability scales.

The relationship between the realization-specific parameter and the abstract representability scale is recorded explicitly in the experiment metadata.

3.5 Common data structures

Each experiment is represented by a machine-readable experiment record containing at least the following fields:

```
experiment_id
prototype
system_size
representability_scale_id
realization_scale_parameter
realization_scale_definition
time
initial_state_id
dynamics_id
random_seed
numerical_precision
algorithm_version
configuration_hash
observable_values
numerical_uncertainties
theoretical_threshold
assumption_status
confinement_status
validation_status
primary_failure_code
secondary_failure_codes
```

The field `realization_scale_parameter` records the numerical value of the prototype-specific control parameter (D_{TN} or D_{QEC}), while `representability_scale_definition` records its interpretation within the corresponding realization. This separation prevents the numerical equality of two scale parameters from being interpreted as equality of their mathematical meaning.

A representability-map object contains:

```
map_id
prototype
scale_D
domain_description
codomain_description
linearity_error
unitality_error
idempotence_error
positivity_status
```

```
complete_positivity_status
construction_method
validation_status
```

An observable record contains:

```
observable_id
state_id
scale_D
raw_value
numerical_error
estimator
optimization_error
convergence_status
```

An assumption-test record contains:

```
assumption_id
prototype
parameter_set
test_statistic
threshold
uncertainty
uncertainty_components
sampling_protocol
optimization_protocol
fitting_protocol
status
failure_mode
diagnostic_data
```

The allowed status values are

$$\{\texttt{PASS}, \texttt{FAIL}, \texttt{INCONCLUSIVE}, \texttt{NOT_TESTED}\}.$$

No result is silently converted from `INCONCLUSIVE` to either `PASS` or `FAIL`.

3.6 TN prototype

3.6.1 Physical model

The baseline TN Hamiltonian is the open-chain qubit Hamiltonian

$$H_{\mathrm{TN}} = J \sum_{j=1}^{N-1} Z_j Z_{j+1} + h_x \sum_{j=1}^{N} X_j + h_z \sum_{j=1}^{N} Z_j.$$

The baseline parameter choice is

$$J = 1, \qquad h_x = 1, \qquad h_z = 0.5.$$

These values define the baseline computational experiment. Simpler integrable or exactly solvable Hamiltonians are retained as validation controls and are not substituted for the baseline model.

3.6.2 State representation

Pure states are represented primarily as matrix-product states,

$$|\psi(T)\rangle,$$

with

$$\rho(T) = |\psi(T)\rangle\langle\psi(T)|.$$

For the smallest system sizes, exact state-vector evolution is performed in parallel and serves as the primary numerical benchmark.

The MPS calculation must record truncation tolerances, retained bond dimensions, discarded weights, and all parameters required to reproduce the state.

3.6.3 TN representability subsystem

The finite-scale TN representability subsystem is generated using an MPO construction with a realization-specific control parameter denoted

$$D_{\mathrm{TN}}.$$

In the baseline implementation,

$$D_{\mathrm{TN}}$$

is the MPO bond-dimension cutoff controlling the finite tensor-network representability scale.

The computational construction is denoted

$$\Pi^{\mathrm{MPO}}_{D_{\mathrm{TN}}},$$

or equivalently

$$\Pi^{\mathrm{MPO}}_{D}$$

when the realization context is explicit.

The parameter D_{TN} is not interpreted as an abstract operator-space dimension unless that correspondence is independently established. Its role is to define the finite computational representability class used by the MPO construction.

The implementation uses a deterministic canonical-form MPO compression or projection procedure with fixed truncation rules.

The computational definition of the map must not be described as a mathematical CP-unital-idempotent projection unless those properties have been independently verified.

In particular, the implementation explicitly tests

$$\Pi^{\mathrm{MPO}}_{D}(I) \simeq I,$$

$$\Pi^{\mathrm{MPO}}_{D} \circ \Pi^{\mathrm{MPO}}_{D} \simeq \Pi^{\mathrm{MPO}}_{D},$$

and

$$\Pi^{\mathrm{MPO}}_{D}(aX + bY) \simeq a\Pi^{\mathrm{MPO}}_{D}(X) + b\Pi^{\mathrm{MPO}}_{D}(Y).$$

Positivity and complete positivity are tested on the finite-dimensional operator sets for which direct numerical verification is feasible.

Failure of these tests is recorded as a failure of the concrete TN realization of the assumed representability structure.

3.6.4 TN dual map

The dual map

$$(\Pi_D^{\mathrm{MPO}})^*$$

is constructed explicitly from the implemented operator map.

For small systems, the dual is obtained by direct matrix representation of the linear map. For larger systems, the corresponding tensor-network contraction is used, with an independently validated small-system implementation.

3.6.5 TN structural observable

The primary TN observable is

$$O_{\mathrm{TN}}(\rho) = \left\| \rho - (\Pi_D^{\mathrm{MPO}})^*(\rho) \right\|_1 .$$

For small systems, the trace norm is evaluated directly.

For larger systems, an explicitly documented approximation is permitted, provided that the approximation error is separately estimated and stored.

The computational output is

$$O_{\mathrm{TN}}(\rho) \pm \delta O_{\mathrm{TN}}.$$

3.6.6 TN assumption test

The primary realization-level compatibility test is

$$O_{\mathrm{TN}}(\rho(T)) \leq \varepsilon_D(T).$$

Define the compatibility margin

$$\Delta_{\mathrm{TN}}(T, D) = O_{\mathrm{TN}}(\rho(T)) - \varepsilon_D(T).$$

The classification rule is

$$\begin{cases} \texttt{PASS}, & \Delta_{\mathrm{TN}} \leq \delta O_{\mathrm{TN}} + \tau_{\mathrm{num}}, \\ \texttt{FAIL}, & \Delta_{\mathrm{TN}} > \delta O_{\mathrm{TN}} + \tau_{\mathrm{num}}, \\ \texttt{INCONCLUSIVE}, & \text{if numerical uncertainty prevents separation.} \end{cases}$$

The result is reported separately for every

$$(N, D, T)$$

rather than replacing finite-size data by a single global conclusion.

3.6.7 TN confinement test

The confinement test asks whether structures satisfying

$$O_{\mathrm{TN}} > \varepsilon_D(T)$$

occur among the computationally generated reachable states.

The project therefore distinguishes between the continuous defect measurement and the empirical reachable-set test.

A candidate structure η is classified as computationally forbidden at (T, D) only if:

1. the TN representability implementation has passed validation;

2. the relevant realization assumptions have passed;

3. the candidate satisfies the forbidden inequality with a numerical margin larger than the uncertainty;

4. the search procedure has adequately sampled the corresponding reachable region.

The final classification is therefore expressed as

$$\texttt{SUPPORTED}, \qquad \texttt{NOT_SUPPORTED}, \qquad \texttt{INCONCLUSIVE}.$$

3.7 QEC prototype

3.7.1 Code family

The baseline QEC realization is the planar surface code with one logical qubit.

The code is indexed by its linear size

$$L,$$

with increasing values

$$L \in \mathcal{L}_{\mathrm{QEC}}$$

chosen to provide a finite-size scaling sequence.

The encoded logical Hilbert space is

$$\mathcal{H}_L \simeq \mathbb{C}^2,$$

and the encoding isometry is

$$V_L : \mathcal{H}_L \to \mathcal{H}_N.$$

3.7.2 Logical states

The baseline logical-state set contains

$$|0_L\rangle, \qquad |1_L\rangle, \qquad |+_L\rangle, \qquad |-_L\rangle,$$

together with selected logical superpositions used to probe structural dependence.

The encoded state is

$$\rho = V_L \rho_L V_L^\dagger.$$

The code-space condition is numerically validated before the state is passed to the dynamical module.

3.7.3 Logical operators

The logical operator algebra is

$$\mathcal{A}_{\log} = V_L \mathcal{B}(\mathcal{H}_L) V_L^\dagger.$$

For the surface-code specialization, the logical Pauli operators

$$\overline{X}, \qquad \overline{Z}$$

are constructed explicitly.

Logical representatives are generated together with their physical support, and the minimum support of a nontrivial logical representative is recorded.

3.7.4 QEC representability scale

The QEC representability scale is controlled by a realization-specific parameter

$$D_{\mathrm{QEC}}.$$

The quantity D_{QEC} is defined as the maximum physical accessibility/support scale admitted into the logical representability construction.

Unlike the TN realization, where the corresponding parameter is an MPO bond-dimension cutoff, D_{QEC} is a locality/accessibility parameter associated with the admissible logical representatives.

The two quantities therefore satisfy the common abstract role

$$D \equiv \text{finite representability scale},$$

but are not numerically identical physical parameters:

$$D_{\mathrm{TN}} \neq D_{\mathrm{QEC}}.$$

Let

$$V_D \subseteq \mathcal{B}(\mathcal{H}_N)$$

denote the span of logical observables possessing admissible representatives at scale D.

The representable subsystem is then

$$\mathcal{A}_D^{(\mathrm{code})} = \mathrm{Alg}(V_D).$$

The implementation does not identify this subsystem with the naive set

$$\{X : \mathrm{wt}(X) \leq D\}.$$

The generated subsystem is constructed explicitly and its closure properties are validated.

3.7.5 QEC representability projection

The computational map is

$$\Pi_D^{(\mathrm{code})} : \mathcal{B}(\mathcal{H}_N) \longrightarrow \mathcal{A}_D^{(\mathrm{code})}.$$

The implementation explicitly tests

$$\Pi_D^2 \simeq \Pi_D,$$

$$\Pi_D(I) \simeq I,$$

linearity, positivity, and complete positivity on the numerically accessible operator domain.

The existence of the desired CP-unital-idempotent projection is therefore treated as a realization condition to be validated, not as an automatic consequence of the definition of the accessible logical operators.

3.7.6 Logical representability defect

The primary QEC observable is

$$O_L(\rho) = \sup_{\substack{X \in \mathcal{A}_{\log} \\ \|X\| \leq 1}} \left| \mathrm{Tr}\left[(X - \Pi_D^{(\mathrm{code})}(X))\rho \right] \right|.$$

For the one-logical-qubit realization, the optimization is reduced to the finite-dimensional logical operator basis and is evaluated to controlled numerical precision.

The resulting value is stored as

$$O_L(\rho) \pm \delta O_L.$$

3.7.7 Logical protection coordinate

For a structural class η, the logical protection coordinate is

$$d_L(\eta) = \min\{\mathrm{wt}(L) : L \in \mathcal{L}_\eta, \quad L \notin \mathbb{C}I\}.$$

The code distance

$$d_{\mathrm{stab}} = \min\{\mathrm{wt}(P) : P \in N(\mathcal{S}) \setminus \mathcal{S}\}$$

is recorded as a code-level reference quantity.

It is not substituted for the structural coordinate $d_L(\eta)$.

3.7.8 Logical representability growth test

The realization-dependent growth assumption is

$$O_L(\eta) \geq g_D(d_L(\eta)).$$

The computational project does not assume a functional form for g_D at the outset.

Instead, the primary procedure constructs the empirical relation

$$\{(d_L(\eta_i), O_L(\eta_i))\}_i$$

and determines whether a reproducible nondecreasing lower envelope exists.

Only after this test has been passed may a candidate function

$$g_D(d)$$

be fitted for the purpose of constructing a finite representability threshold.

A candidate g_D must satisfy the empirical requirement

$$O_L(\eta_i) \geq g_D(d_L(\eta_i))$$

within the declared numerical uncertainty for the validated data set.

Failure to establish a monotone lower bound is recorded as failure of the logical representability growth assumption for that realization and scale.

3.7.9 QEC observable-level A6 test

The second realization-dependent condition is

$$O_L(\rho(T)) \leq \varepsilon_D(T).$$

Define

$$\Delta_L(T, D) = O_L(\rho(T)) - \varepsilon_D(T).$$

The same PASS/FAIL/INCONCLUSIVE logic used for the TN prototype is applied.

3.7.10 QEC representable logical scale

Once the growth assumption has been validated, define

$$d_{\max}(T, D) = \max\{d \in \mathbb{N} : g_D(d) \leq \varepsilon_D(T)\}.$$

If the set is empty, the result is recorded explicitly rather than assigning an artificial value.

The threshold is recomputed for every

$$(T, D)$$

for which a valid g_D has been established.

3.7.11 QEC confinement test

The predicted forbidden region is

$$d_L(\eta) > d_{\max}(T, D).$$

The primary confinement test therefore compares the empirically reachable structural set with the predicted forbidden region.

The result is classified as:

`SUPPORTED`

if the validated implementation and realization assumptions hold and the predicted forbidden structures are consistently absent from the sampled reachable set;

`NOT_SUPPORTED`

if validated reachable structures violate the predicted confinement with a numerical margin exceeding the uncertainty;

and

`INCONCLUSIVE`

if finite-size limitations, sampling limitations, or numerical uncertainty prevent a reliable distinction.

3.8 Common algorithms

3.8.1 Algorithm: experiment initialization

For each experiment:

1. load the frozen configuration;
2. validate all parameter ranges;
3. assign a unique experiment identifier;
4. record the software and configuration hashes;
5. initialize the numerical precision;
6. initialize the random seed if stochastic operations are present;
7. construct the physical model;
8. construct the initial state;
9. execute the state-validation tests;
10. construct the representability map;
11. execute the representability-map validation tests.

No observable or confinement calculation is executed if a mandatory initialization or representability validation test fails.

3.8.2 Algorithm: observable evaluation

For each state $\rho(T)$:

1. construct or retrieve the validated representability map;
2. construct the dual map where required;
3. evaluate the structural observable;
4. estimate numerical error;
5. repeat at increased numerical accuracy when required;
6. store the converged value and uncertainty;
7. associate the result with the complete experiment metadata.

3.8.3 Algorithm: assumption testing

For every realization-dependent assumption:

1. identify the theoretical inequality;
2. compute the measured quantity;
3. compute the theoretical threshold;
4. calculate the residual or margin;
5. calculate the total numerical uncertainty;
6. classify the result as PASS, FAIL, or INCONCLUSIVE; the classification is subject to the global failure-precedence rule defined in Section 3.14. In particular, a numerical failure overrides an assumption classification, and an implementation failure overrides all subsequent assumption and confinement classifications.
7. store the complete diagnostic record.

3.8.4 Algorithm: confinement testing

A confinement test is executed only if all mandatory numerical, implementation, and realization-assumption tests for the corresponding realization have passed. The failure-precedence rule defined in Section 3.14 determines the primary experiment classification whenever multiple failure conditions are detected.

For every candidate structure:

1. evaluate its structural coordinate;
2. evaluate the corresponding representability observable;
3. evaluate the predicted threshold;
4. determine whether the candidate lies inside or outside the predicted reachable region;
5. compare the prediction with the empirical reachable-set data;
6. classify the result as SUPPORTED, NOT_SUPPORTED, or INCONCLUSIVE.

3.9 Numerical decision procedures

All numerical decision procedures used to classify assumptions and confinement outcomes are fixed before production experiments are executed. The purpose of this specification is to prevent analysis-dependent changes to fitting, sampling, optimization, or uncertainty procedures after observing numerical results.

3.9.1 Construction of the logical growth function

For the QEC prototype, the empirical lower envelope associated with the logical representability growth condition is constructed using a fixed fitting protocol.

Given the measured data set

$$\mathcal{D}_D = \{(d_L(\eta_i), O_L(\eta_i), \delta O_L(\eta_i))\}_i ,$$

the candidate growth function

$$g_D(d)$$

is obtained by a constrained monotone fit satisfying

$$g_D(d_{i+1}) \geq g_D(d_i).$$

The default fitting procedure is monotone regression with uncertainty-weighted least squares. The fitted function minimizes

$$\sum_i w_i \left(O_L(\eta_i) - g_D(d_L(\eta_i))\right)^2 ,$$

where

$$w_i = \frac{1}{(\delta O_L(\eta_i))^2 + \tau_{\text{num}}^2}.$$

The fitted function is accepted only if

$$O_L(\eta_i) \geq g_D(d_L(\eta_i)) - (\delta O_L(\eta_i) + \tau_{\text{num}})$$

for every validated data point.

If no monotone lower envelope satisfying this condition exists, then the logical representability growth assumption is classified as

`FAIL`.

The fitting procedure, weighting rule, and acceptance criterion are fixed before evaluation of confinement results.

3.9.2 Sampling strategy

All reachable-set and structural-data searches use a predefined sampling protocol.

For each experiment point

$$(N, D, T)$$

or

$$(L, D, T),$$

the sampled ensemble is generated according to a fixed distribution specified in the experiment configuration.

The configuration records

```
sampling_method
sample_count
initial_state_distribution
random_seed
coverage_parameters
stopping_criterion
```

The sampling procedure is considered converged only if the relevant observables are stable under an independent increase of the sample count.

The sampling convergence criterion is

$$|Q_{n_{\text{high}}} - Q_{n_{\text{low}}}| \leq \tau_{\text{sample}}.$$

Failure of sampling convergence results in

```
INCONCLUSIVE
```

rather than

```
PASS
```

or

```
FAIL.
```

3.9.3 Optimization tolerances

Every optimization-based observable evaluation uses explicitly recorded numerical tolerances.

The optimization configuration records

```
optimizer
absolute_tolerance
relative_tolerance
maximum_iterations
termination_condition
```

For an optimization procedure producing value Q, convergence requires

$$|Q_{k+1} - Q_k| \leq \tau_{\text{opt}}.$$

If convergence cannot be established within the declared tolerance, the observable is classified as numerically unresolved and the associated test is

```
INCONCLUSIVE.
```

3.9.4 Uncertainty aggregation

All reported uncertainties are aggregated before comparison with theoretical thresholds.

For a measured observable

$$Q_{\text{meas}},$$

the total numerical uncertainty is defined as

$$\delta Q_{\mathrm{tot}} = \sqrt{\delta Q_{\mathrm{round}}^2 + \delta Q_{\mathrm{trunc}}^2 + \delta Q_{\mathrm{disc}}^2 + \delta Q_{\mathrm{opt}}^2 + \delta Q_{\mathrm{sample}}^2}.$$

All hypothesis tests use

$$Q_{\mathrm{meas}} \pm \delta Q_{\mathrm{tot}}$$

together with the declared numerical tolerance

$$\tau_{\mathrm{num}}.$$

No uncertainty contribution may be removed after observing the resulting classification.

3.10 No-circularity rule for threshold construction

The construction of any representability threshold used in a confinement test must be independent of the confinement outcome.

In particular, the procedure used to estimate

$$\varepsilon_D(T)$$

or

$$d_{\max}(T, D)$$

must not use information obtained from the subsequent reachable-set or forbidden-region classification.

The computational pipeline therefore enforces the ordering

implementation validation → observable evaluation → threshold construction → confinement test.

The confinement outcome is evaluated only after the threshold has been fixed.

No fitting parameter, uncertainty estimate, sampling choice, optimization criterion, or model-selection step used in threshold construction may be modified using knowledge of whether candidate structures satisfy or violate the predicted confinement condition.

For the TN prototype, the compatibility threshold

$$\varepsilon_D(T)$$

is treated as an independently specified theoretical or realization-dependent quantity and is not adjusted using observed values of

$$O_{\mathrm{TN}}(\rho(T)).$$

For the QEC prototype, the growth function

$$g_D(d)$$

and the resulting threshold

$$d_{\max}(T, D)$$

are constructed exclusively from the validated logical representability data and numerical decision protocol defined prior to confinement evaluation.

Any analysis procedure in which threshold construction depends on the observed confinement result is classified as invalid due to circularity.

Such a result cannot be used as computational evidence for or against the proposed confinement mechanism.

3.11 Experiment matrix

The baseline experiment matrix is

$$\mathcal{X}_{\mathrm{TN}} = \mathcal{N}_{\mathrm{TN}} \times \mathcal{D}_{\mathrm{TN}} \times \mathcal{T} \times \mathcal{S}_{\mathrm{TN}},$$

where $\mathcal{N}_{\mathrm{TN}}$ is the system-size set, $\mathcal{D}_{\mathrm{TN}}$ the MPO representability scales, $\mathcal{T}$ the time grid, and $\mathcal{S}_{\mathrm{TN}}$ the initial-state set.

The QEC experiment matrix is

$$\mathcal{X}_{\mathrm{QEC}} = \mathcal{L}_{\mathrm{QEC}} \times \mathcal{D}_{\mathrm{QEC}} \times \mathcal{T} \times \mathcal{S}_{\mathrm{QEC}}.$$

The minimum baseline matrix contains:

- at least four increasing TN system sizes;
- at least three TN representability scales;
- at least four increasing QEC code sizes;
- at least three QEC representability scales;
- a common time grid;
- reference and structured initial-state classes;
- exact small-system validation runs.

A full parameter sweep is not required before the implementation and assumption tests have passed.

3.12 Validation and reproducibility tests

3.12.1 State validation

Every state must satisfy

$$\rho^\dagger \simeq \rho, \qquad \rho \geq 0, \qquad \mathrm{Tr}\,\rho \simeq 1.$$

For pure-state calculations,

$$\langle \psi | \psi \rangle \simeq 1.$$

For QEC states, code-space membership is additionally verified.

3.12.2 Representability-map validation

Every implemented representability map must be tested for:

$$\text{linearity}, \quad \text{unitality}, \quad \text{idempotence}, \quad \text{positivity}, \quad \text{complete positivity},$$

to the extent numerically accessible.

The numerical deviations

$$\delta_{\mathrm{lin}}, \quad \delta_{\mathrm{unital}}, \quad \delta_{\mathrm{idemp}}, \quad \delta_{\mathrm{pos}}, \quad \delta_{\mathrm{CP}}$$

are stored explicitly.

3.12.3 Convergence tests

Every primary observable must be tested against the relevant numerical control parameter.

For TN calculations this includes, as appropriate,

$$\chi_{\mathrm{MPS}}, \qquad \delta_{\mathrm{MPO}}, \qquad \text{time-step tolerance.}$$

For QEC calculations this includes the precision of the operator optimization and any numerical linear-algebra tolerance.

A result is considered converged only if

$$|Q_{\mathrm{high}} - Q_{\mathrm{low}}| \leq \tau_{\mathrm{conv}}$$

for the declared convergence tolerance.

3.12.4 Exact small-system benchmark

For the smallest system sizes, all quantities that admit exact evaluation must be compared with an independent exact implementation.

The benchmark must include:

$$\text{state evolution}, \qquad \text{representability map}, \qquad \text{observable}, \qquad \text{structural coordinate},$$

where applicable.

For the QEC prototype specifically, extracted logical operators must additionally be compared against the analytically known logical operators of the corresponding code family (e.g. the standard surface-code $\overline{X}$, $\overline{Z}$ representatives), independent of the computational extraction procedure.

Independent external reference implementations. The independent exact implementation required above may be realized by hand-written dense linear algebra and, where useful, additionally by established open-source packages used purely as external reference implementations. These packages are not part of the production pipeline, and no production result depends on them. A package qualifies as an independent reference only if it shares no code with the project's compiled kernels. Subject to the license and capability audit of Section 1.1.12, the candidates are:

- *Qiskit Aer, stabilizer method (Prototype B).* Aer's stabilizer simulation method executes Clifford circuits, which is the formalism of the stabilizer-code baseline (Section 3.7): the encoding circuits, stabilizer generators, and Pauli logical-operator representatives of the planar surface code are Clifford/Pauli objects. For the smallest code sizes it is a candidate for independently regenerating the encoded stabilizer states and their stabilizer generators, and for sampling Pauli measurements of the logical operators, for comparison with the project's stabilizer-tableau kernels and, where adopted, with Stim. This validates the stabilizer, code-space, and logical-operator layers only. The representability map and O_L are not in general Clifford quantities and remain validated by the exact implementation above.

- *QuEST (either prototype).* QuEST is a C/C++ state-vector and density-matrix simulator with OpenMP/MPI parallelism; its compiled-kernel design parallels the execution model of Section 1.1.9. It is a candidate further independent small-system reference implementation: for Prototype A, for small-system state evolution and observables; for Prototype B, for small code sizes at which the full state vector is tractable.

- *QuTiP (minor, optional).* QuTiP may serve as one further optional cross-check for small closed systems, using its closed-system solvers only. Its principal strength, open-quantum-system and dissipative dynamics, is outside the scope of this project and is not used; QuTiP therefore has a minor supporting role, not a methodological one.

Agreement between an external reference and the project implementation is judged at the numerical tolerance declared for the corresponding quantity; a disagreement beyond that tolerance is a validation failure in the sense of Section 3.14. The use of these packages does not extend the project to supporting every quantum software stack (Section 2.2).

3.12.5 Reproducibility tests

Every production result must be reproducible from:

$$\{\text{source version}, \text{configuration}, \text{random seed}, \text{software environment}, \text{input data}\}.$$

The configuration file and environment specification are archived with the results.

The archived decision record must also contain the pre-confinement threshold construction data and procedures, allowing verification that the threshold was fixed independently of the confinement outcome.

The numerical decision procedures are archived together with the results, including:

$$\{\text{fit procedure}, \text{sampling configuration}, \text{optimization tolerances}, \text{uncertainty aggregation rule}\}.$$

A published numerical conclusion must therefore be reproducible not only from the raw data and software environment, but also from the complete decision protocol used to classify the result.

Repeated runs using the same deterministic configuration must produce identical results to the declared numerical tolerance.

3.13 Acceptance criteria

The project uses separate acceptance criteria for implementation, assumptions, and confinement.

3.13.1 Implementation acceptance

A prototype is implementation-valid only if:

1. the physical states pass normalization and positivity tests;
2. the evolution passes independent small-system benchmarks;
3. the representability map passes the required structural tests within declared numerical tolerance;
4. the structural observable is numerically converged;
5. all required metadata are recorded.

Failure of any mandatory condition results in

`IMPLEMENTATION_FAIL`

for that configuration.

3.13.2 Assumption acceptance

An additional realization assumption is considered supported only if:

1. the relevant implementation tests have passed;
2. the empirical relation is observed over the prescribed parameter range;
3. the relation is stable under numerical convergence tests;
4. the result is not removed by increasing system size or numerical precision within the tested range;
5. the separation from the acceptance threshold exceeds the declared numerical uncertainty.

Otherwise the assumption is classified as

`FAIL`

or

`INCONCLUSIVE`.

3.13.3 Confinement acceptance

A confinement claim is computationally supported only if:

1. implementation validation has passed;
2. all realization assumptions required for the prediction have passed;
3. the forbidden-region criterion is satisfied with a margin larger than numerical uncertainty;
4. the result persists over the relevant finite-size and representability-scale sequence;
5. control calculations do not reproduce the same apparent exclusion for trivial numerical reasons.
6. The threshold construction and decision procedure are independent of the observed confinement result.

The computational conclusion is then stated as

computational support for the proposed confinement mechanism

rather than as a proof of the theoretical theorem.

3.14 Failure taxonomy

The project records failures according to the following taxonomy.

Code	Meaning
I-STATE	Physical or encoded state construction failed validation.
I-DYN	Numerical dynamics failed convergence or exact benchmark validation.
I-REP	The proposed representability map failed the required structural tests.
I-OBS	The structural observable could not be evaluated to the required precision.
A-FIBRE	The assumed structural fibre invariance was not established.
A-A6	The observable-level compatibility estimate was not supported.
A-GROWTH	The realization-dependent representability-growth assumption was not supported.
C-FAIL	Validated assumptions hold, but the predicted confinement is not observed.
C-INCONCLUSIVE	The available numerical evidence is insufficient to classify the confinement prediction.
M-CIRC	Threshold construction or decision procedure depended on the confinement outcome, invalidating the computational classification.
NUM	Numerical precision, convergence, or conditioning prevents reliable classification.

3.14.1 Failure precedence

When multiple failure conditions occur within the same experiment, the final classification is determined by a fixed precedence order. The highest-priority failure masks all lower-priority failure classifications for purposes of the primary experiment status.

The precedence order is

NUM > I-STATE/I-DYN/I-REP/I-OBS > A-FIBRE/A-A6/A-GROWTH > C-FAIL/C-INCONCLUSIVE.

Equivalently, the computational decision hierarchy is

numerical validity
↓
implementation validity
↓
realization-assumption validity
↓
confinement outcome.

A numerical failure takes precedence over every other classification because the corresponding numerical result cannot be regarded as reliable evidence.

An implementation failure takes precedence over any realization-assumption or confinement classification because the latter cannot be validly evaluated using an invalid computational realization.

A realization-assumption failure takes precedence over confinement failure because the confinement prediction is conditional on the corresponding assumption.

A confinement failure or inconclusive result is therefore assigned only when the numerical implementation and all realization assumptions required for the confinement test have passed.

All lower-priority failures detected during the same experiment are nevertheless retained in the diagnostic record. Failure precedence determines only the primary experiment classification and does not delete secondary diagnostics.

This taxonomy is part of the scientific output and is not merely an internal software diagnostic.

3.15 Output specification

The computational project produces six classes of output.

3.15.1 Raw computational data

Raw data include:

- physical states or their reproducible compressed representations;
- Hamiltonian parameters;
- time grids;
- representability scales;
- logical operators and code data;
- random seeds;
- numerical tolerances;
- intermediate representability-map data.

3.15.2 Derived observables

The derived-data layer contains:

$$O_{\mathrm{TN}}(\rho(T)),$$
$$O_L(\rho(T)),$$
$$d_L(\eta),$$
$$g_D(d),$$
$$\varepsilon_D(T),$$

and

$$d_{\max}(T, D).$$

Each quantity is accompanied by numerical uncertainty and provenance.

3.15.3 Validation reports

For every prototype the project produces a validation report containing:

- state-validation results;
- evolution-convergence results;
- representability-map validation;
- observable convergence;
- exact small-system comparison;
- reproducibility checks.

3.15.4 Assumption-status tables

A machine-readable and human-readable table is generated with the structure

$$\text{Assumption} \mid \text{Observable} \mid \text{Criterion} \mid \text{Measured} \mid \text{Status}.$$

Separate tables are generated for the TN and QEC realizations.

3.15.5 Confinement results

The final confinement output contains:

- measured structural observables;
- predicted thresholds;
- empirical reachable structures;
- predicted forbidden structures;
- finite-size dependence;
- representability-scale dependence;
- uncertainty margins;
- final classification.

The primary graphical outputs are:

1. O_{TN} versus T for several D;
2. O_{TN} versus $\varepsilon_D(T)$;
3. TN compatibility margins;
4. O_L versus d_L for several D;
5. empirical lower envelopes used to construct g_D;
6. $d_{\max}(T, D)$ versus D;
7. reachable versus predicted-forbidden logical structures;
8. finite-size scaling plots;
9. convergence and validation diagnostics.

3.16 Reproducibility package

Each repository (Section 4) maintains its own reproducibility package.

For the TN and QEC repositories, the final computational release contains:

```
configs/
data/
|   +-- raw/
|   +-- processed/
|   +-- validation/
experiments/
```

```
results/
|   +-- tables/
|   +-- figures/
|   +-- reports/
tests/
|   +-- unit/
|   +-- integration/
|   +-- exact/
|   +-- convergence/
environment/
documentation/
README
```

The framework repository does not itself run experiments and therefore does not maintain `configs/`, `data/`, `experiments/`, or `results/`. It maintains the same reproducibility discipline restricted to `tests/`, `environment/`, `documentation/`, and `README`, applied to the generic modules it exposes (Section 3.3).

Every published numerical result must be traceable from a final figure or table to:

$$\boxed{\text{configuration} \to \text{experiment} \to \text{raw data} \to \text{analysis} \to \text{figure/table.}}$$

The archived `environment/` specification records the exact version of every software component, including any external reference implementation used for the exact small-system benchmark (Section 3.12.4). Any result obtained through the optional GPU-acceleration path (Section 1.1.10) is labeled as such in its configuration, tables, and figures, and is accompanied by the corresponding CPU-path result wherever both are feasible.

3.17 Final scientific deliverables

The computational project produces the following final scientific deliverables.

1. A validated TN prototype implementing the finite-scale MPO representability construction.
2. A validated QEC prototype implementing the finite-scale logical representability construction.
3. A complete implementation-status report documenting whether the representability maps satisfy the structural properties required by the corresponding theoretical construction.
4. An assumption-testing report documenting the empirical status of every realization-dependent assumption.
5. A TN confinement report testing
$$O_{\mathrm{TN}} > \varepsilon_D(T)$$
as the proposed forbidden-region criterion.
6. A QEC confinement report testing
$$d_L > d_{\max}(T, D)$$
as the proposed forbidden-region criterion.
7. A complete numerical uncertainty and finite-size analysis.

8. A machine-readable archive containing raw and processed computational data.
9. A reproducibility package containing source code, configurations, environment information, seeds, validation tests, and analysis scripts.
10. A final scientific classification for each realization:

$$\boxed{\text{supported,} \quad \text{unsupported,} \quad \text{or} \quad \text{inconclusive.}}$$

3.18 Interpretation of computational outcomes

The computational project deliberately distinguishes four logically different scientific outcomes.

Implementation failure. If the required representability structure cannot be implemented with the properties assumed by the theoretical construction, the result is reported as a failure of the selected computational realization. No confinement claim is subsequently tested using that invalid implementation.

Assumption failure. If the representability implementation is valid but a realization-dependent assumption such as

$$O_L \geq g_D(d_L)$$

or

$$O_{\mathrm{TN}} \leq \varepsilon_D(T)$$

is not supported, the computational result identifies the corresponding assumption as the point at which the proposed mechanism fails in the chosen realization.

Such a result does not constitute a repair of the theory and is not interpreted as a general refutation of the abstract framework.

Confinement failure. If all required implementation and realization assumptions pass but the predicted forbidden structures are nevertheless observed among the computationally reachable structures, the project records a failure of the predicted computational consequence for that realization and parameter regime.

Confinement support. If implementation validation and realization-assumption tests pass, and the predicted forbidden region is consistently excluded from the reachable computational data, the result provides numerical support for the proposed confinement mechanism in that realization.

The conclusion remains conditional:

$$\boxed{\text{validated realization assumptions} + \text{observed confinement} \Longrightarrow \text{computational support.}}$$

It is not a numerical proof of the abstract theorem.

3.19 Claims not established by the project

The project does not establish:

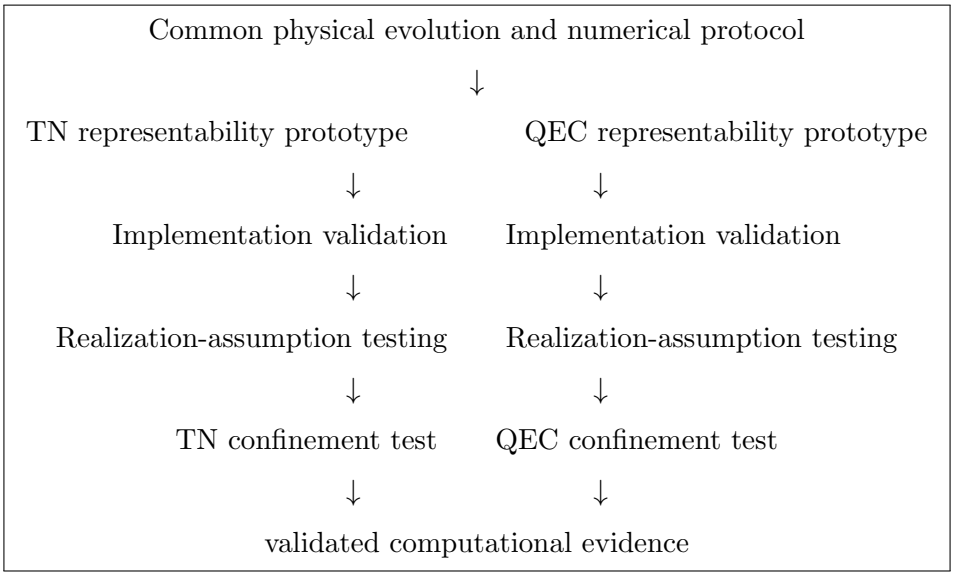
- the universal validity of the abstract theorem beyond the tested realizations;
- that every tensor-network family satisfies the required growth condition;
- that every QEC family satisfies the logical representability growth condition;

- that all possible operator subsystems admit the required completely-positive projection;
- a universal physical bound on quantum computation;
- impossibility of large-distance quantum codes;
- impossibility of arbitrarily expressive tensor networks;
- quantum computational advantage;
- asymptotic optimality of either implementation;
- a complete experimental realization of all Axioms A1–A6.

These exclusions are scientifically important because they prevent the computational results from being overstated.

3.20 Summary of the computational design

The complete computational project is therefore organized as



The central methodological principle is that every theoretical statement used by the computational project is assigned one of three roles:

implemented,	tested,	conditional.

The computational project does not silently promote a conditional theoretical assumption into an implementation fact. In particular, the existence and properties of the concrete representability projections, the logical representability growth condition, and the observable-level realization of Axiom A6 are treated as explicit computationally testable ingredients.

This separation allows the computational project to identify precisely whether an observed discrepancy originates in numerical implementation, in a realization-dependent assumption, or in the predicted confinement mechanism itself.

4 Repository Architecture

The project is organized as three independent repositories:

A. `quantum-confinement-framework`: a lightweight shared repository containing only the generic, domain-agnostic components of the common software architecture (Section 3.3);

B. `quantum-confinement-tn`: the tensor-network realization (Prototype A) and its associated numerical experiments;

C. `quantum-confinement-qec`: the quantum-error-correction realization (Prototype B) and its associated numerical experiments.

The dependency direction is strictly one-way:

$$\texttt{quantum-confinement-framework} \longrightarrow \begin{cases} \texttt{quantum-confinement-tn}, \\ \texttt{quantum-confinement-qec}. \end{cases}$$

Neither realization depends on the other, and neither the TN nor the QEC repository contributes realization-specific code back into the framework repository. Both prototype repositories may import the generic interfaces exposed by the framework repository, but the scientific implementation of each realization – state construction, representability maps, observables, and confinement tests – remains local to its own repository.

4.1 Scope of the shared framework repository

During the prototype phase, the framework repository is deliberately restricted to the generic subset of the common software architecture (Section 3.3): configuration, experiment orchestration, validation utilities, scaling and error-analysis utilities, and reproducibility support.

It must not become a universal tensor-network library, a universal quantum-error-correction library, a common quantum simulator, or a common HPC execution engine, and it must not host any realization-specific numerical kernel.

share infrastructure, not scientific mechanisms.

4.2 Repository independence and future expansion

The three-repository architecture is intended to support expansion beyond the two prototypes described in this specification. A future realization can be added as an independent repository,

`quantum-confinement-<new-realization>`,

that depends only on `quantum-confinement-framework`, without requiring any modification to the existing TN or QEC repositories.

The repository architecture therefore mirrors the conceptual structure of the project:

common confinement framework + independent physical realizations.

This separation is particularly important for the present work, since the TN and QEC prototypes are intended to provide computationally distinct validations of the same abstract confinement framework, not two implementations of a common numerical model.

5 Consistency and Traceability Audit of the Computational Specification

The formal computational specification is subject to a consistency and traceability audit before implementation-specific details are finalized. The purpose of this audit is to verify that every computational design choice required by the project has a traceable relationship to the scientific

objective, the corresponding theoretical or realization assumption, the computational module that implements it, the experiment used to test it, the acceptance criterion applied to the result, and the output produced by the computational project.

This audit is not intended to repair, strengthen, or reinterpret the underlying theoretical framework. In particular, a statement that is not established by the theoretical development is not promoted to a computational fact merely because it is required by the implementation. Instead, such a statement is explicitly recorded as an additional realization assumption and, where possible, subjected to a dedicated computational test.

The resulting traceability structure is

$$\text{theoretical statement} \longrightarrow \text{computational design choice} \longrightarrow \text{MCS module} \longrightarrow \text{experiment} \longrightarrow \text{test} \longrightarrow a$$

The audit therefore provides the final consistency check between the scientific specification and the executable computational project.

5.1 Realization hierarchy

The computational interpretation of the theoretical framework follows a strict realization hierarchy. Each level must be validated before the computational evidence at the next level is interpreted.

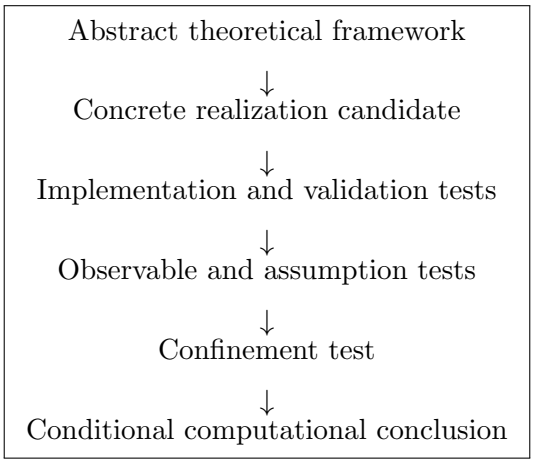


Figure 1: Realization hierarchy for the computational project. The abstract framework is instantiated by a concrete TN or QEC realization, which must first pass implementation and validation tests. Only then are the relevant structural observables and realization-dependent assumptions evaluated. Only after these conditions have been satisfied is the confinement prediction tested.

The hierarchy imposes a one-way logical dependency. Failure at a higher level prevents the corresponding lower-level claim from being interpreted as evidence for the proposed confinement mechanism.

In particular,

$$\begin{gathered}\text{failed implementation} \implies \text{no valid assumption test} \\ \text{failed realization assumption} \implies \text{no conditional confinement claim} \\ \text{validated implementation} + \text{validated assumptions} \implies \text{confinement test permitted.}\end{gathered}$$

The hierarchy applies independently to the TN and QEC prototypes. The realization-specific definitions of the representability map, structural observable, and realization assumptions are supplied by the corresponding prototype sections.

5.2 Audit status conventions

Each design item is assigned one of the following audit statuses.

C — Complete. The scientific choice, computational implementation, test, and acceptance criterion are sufficiently specified for implementation.

T — Traceable but requiring explicit documentation. The underlying computational choice has already been settled, but the choice must be stated explicitly in the final computational specification or implementation documentation.

A — Assumption-dependent. The computational procedure is specified, but its interpretation depends on an additional realization assumption that is not derived from Axioms A1–A6. The assumption must therefore be tested or explicitly reported as a conditional premise.

G — Genuine specification gap. A concrete implementation-level choice required to execute the experiment has not yet been fixed. Such a gap must be resolved before the corresponding computational module is frozen.

N/A — Not applicable. The item does not require a separate computational instantiation for the prototype under consideration.

The distinction between an assumption-dependent item and a genuine specification gap is essential. A realization assumption is not treated as a missing implementation choice. Rather, it becomes an explicit object of computational testing.

5.3 Traceability of the six computational-design blocks

The six computational-design blocks established for the project are mapped onto the formal MCS as follows.

The six blocks therefore form a complete computational specification hierarchy: Blocks 1–3 define what is implemented, Block 4 defines how the scientific claims are tested, Block 5 defines how computational validity is established, and Block 6 defines what constitutes the resulting computational record.

5.4 Traceability of the thirteen blocking design decisions

The thirteen previously identified blocking decisions are treated as implementation-freeze decisions. The recommended choices previously adopted for the project are therefore taken as fixed for purposes of the present audit.

The table is the principal implementation-freeze record for the computational project. No design decision listed as "C" is to be changed during implementation without explicitly recording the change as a revision of the computational specification.

5.5 Theory–assumption–test traceability

The computational project distinguishes three logically different categories of statements.

First, some properties are inherited directly from the abstract framework and are therefore treated as framework-level premises.

Table 2: Traceability of the six computational-design blocks.

No.	Design block	MCS realization	Primary verification	Status
1	Freeze the common experimental protocol	Common configuration, initialization, parameter conventions, seeds, scaling rules, and execution protocol	Protocol validation and configuration audit	C/T
2	Define the TN prototype	TN model, representability construction, structural observable, scaling experiments, controls, and failure diagnostics	TN implementation and numerical validation	C/A
3	Define the QEC prototype	Code instance, logical operators, representability subsystem, projection, logical defect, distance coordinate, and decision tree	QEC realization and assumption tests	C/A
4	Define the hypothesis-testing logic	Null/alternative hypotheses, controls, acceptance criteria, and failure branches	Automated hypothesis tests	C
5	Specify validation and reproducibility	Independent checks, convergence tests, random seeds, metadata, configuration records, and reproducibility outputs	Reproducibility and validation suite	C/T
6	Decide the project outputs	Numerical data, figures, tables, diagnostics, logs, configuration files, and summary report	Output completeness audit	C

Table 3: Master traceability audit for the thirteen blocking computational design decisions.

No.	Decision	Frozen computational choice	MCS realization	Verification / test	Status
1	Scientific computational objective	Implementation → assumption testing → confinement test → failure diagnosis	Common project controller and experiment registry	Objective-level acceptance test	C
2	Common experimental protocol	One common parameter/configuration schema and common execution, logging, seeding, and reporting protocol	Common framework module	Configuration and protocol audit	C/T
3	TN prototype target	Finite TN/MPO realization of the representability structure and its structural observable	TN module and TN experiment suite	Representation, convergence, and scaling tests	C/A
4	QEC prototype target	Concrete encoded-code realization with logical operators and finite logical representability	QEC module and QEC experiment suite	Encoding, logical-operator, and representability tests	C/A
5	Representability scale	Finite computational scale D is the controlled representability parameter	Common parameter object and prototype-specific interpretation	Scale-sweep experiments	C
6	Structural observable	Use the prototype-specific representability defect as the computational structural observable	Observable interface shared by TN and QEC modules	Boundedness, consistency, and structural invariance tests	C/A
7	Reference / control construction	Each confinement experiment includes a control in which the relevant representability restriction is absent, relaxed, or otherwise neutralized	Control experiment module	Control-vs-target comparison	C
8	Assumption-testing strategy	Additional realization assumptions are tested independently before the conditional confinement claim is evaluated	Assumption-test stage in experiment pipeline	Pass/fail assumption tests	C
9	Confinement hypothesis	Test whether structures exceeding the predicted representability threshold are excluded or suppressed according to the specified observable criterion	Confinement-test module	Comparison with predicted threshold $d_{\max}(T, D)$ or TN analogue	C/A
10	Failure-diagnosis logic	If an assumption fails, identify the first failed assumption and do not interpret the subsequent confinement test as confirmation of the theorem	Decision tree and failure-report module	Automated failure classification	C
11	Numerical validation	Convergence, finite-size, numerical stability, normalization, and independent consistency checks are mandatory, including external reference implementations at Level 2 (Section 3.12.4)	Validation module	Validation test suite	C/T
12	Reproducibility	All parameters, seeds, software configuration, experiment identifiers, and generated data are recorded; baseline results use free/open-source software only, and the optional GPU path (Section 1.1.10) is the sole labeled exception	Metadata, configuration, and run manifest modules	Re-run and independent reproduction tests; GPU-path results agree with CPU-path results within tolerance	C/T
13	Project outputs	Machine-readable data plus figures, tables, diagnostics, acceptance reports, and complete run metadata constitute the computational record	Output manager and archive structure	Output-completeness audit	C

Second, some properties are additional realization assumptions required to instantiate the abstract framework in the TN or QEC prototype.

Third, some properties are empirical computational consequences that are tested only after the relevant realization assumptions have been evaluated.

The computational logic is therefore

$$\boxed{\text{framework premise} \neq \text{realization assumption} \neq \text{numerical consequence.}}$$

For the two principal prototypes, the traceability structure is

Table 4: Traceability of realization assumptions to computational tests.

Prototype	**Realization assumption**	**Computational test**	**Interpretation if failed**
TN	The selected finite TN/MPO construction faithfully realizes the intended finite representability subsystem	Representation and convergence tests, including scale dependence	The TN realization does not establish the corresponding abstract representability claim at the tested scale
TN	The chosen structural observable is stable under the prescribed representation and numerical approximation	Observable consistency and convergence tests	The observable is not a sufficiently robust computational proxy for the intended structural quantity
QEC	The selected logical operator subsystem satisfies the required representability structure	Operator-subsystem and projection tests	The chosen QEC realization does not instantiate the assumed representability structure
QEC	The logical representability growth condition holds for the tested code family and parameter range	Measurement of the logical defect versus d_L and comparison with g_D	The confinement conclusion is not supported conditionally for that parameter regime
QEC	The observable-level realization of Axiom A6 holds	Direct measurement of O_L against $\varepsilon_D(T)$	The Axiom A6-to-observable identification fails for the selected realization
Both	The numerical implementation is converged and reproducible	Independent reruns, convergence tests, and metadata verification	The corresponding numerical result is not accepted as computational evidence

The failure of any realization assumption does not constitute a failure of the underlying theoretical framework. It instead establishes that the selected computational realization does not satisfy the conditions required for the conditional confinement test.

5.6 Hypothesis-testing traceability

The project does not use the phrase "the theorem is confirmed" as a generic computational outcome. Instead, each experiment is classified according to the logical status of the relevant assumptions.

The decision structure is

Implementation valid?
↓
Realization assumptions pass?
↓
Predicted confinement observed?

More explicitly:

1. If the implementation fails its validation criteria, the experiment is classified as *computationally invalid*.
2. If the implementation passes but a required realization assumption fails, the experiment is classified as *assumption failure*.
3. If the implementation and all required realization assumptions pass, the confinement hypothesis is tested.
4. If the confinement test passes its predefined acceptance criterion, the result is classified as *conditional computational support* for the corresponding realization.
5. If the confinement test fails despite all preceding conditions passing, the result is classified as *conditional confinement failure* and is reported without modifying the theoretical framework.

Thus the computational project explicitly separates

implementation validity, assumption validity, confinement evidence.

5.7 Final implementation-freeze criterion

The computational design is considered frozen only when every item required by the six computational-design blocks and the thirteen blocking decisions has one of the following statuses:

C, T, A

and no unresolved item remains with status G.

Items classified as T require explicit documentation in the implementation or configuration specification but do not require a new scientific design decision.

Items classified as A are deliberately retained as conditional scientific assumptions and must be connected to an explicit computational test.

Only a status G constitutes a genuine blocker to implementation.

Accordingly, the final computational specification obeys the principle

No hidden design choices and no unacknowledged theoretical assumptions.

The purpose of the present audit is therefore not merely organizational. It establishes the traceability required for the computational results to be interpretable as evidence about the proposed realizations without silently repairing or strengthening the theoretical framework.

6 Potential Follow-Up Technical Papers

The six-week prototypes are deliberately designed so that they can later serve as the computational foundation for two independent technical papers.

6.1 TN paper

A potential paper would focus on:

> *Computational observation of finite tensor-network representability confinement through structural observables and scaling.*

The paper would emphasize:

- the TN-specific structural observable;
- the finite representability construction;
- scaling behavior;
- numerical obstruction;
- HPC implementation.

6.2 QEC paper

A separate paper would focus on:

> *Logical representability obstruction in quantum error-correcting codes: a computational realization of finite logical structural confinement.*

The paper would emphasize:

- logical-operator extraction;
- finite logical representability;
- O_L;
- logical-protection scaling;
- numerical obstruction;
- HPC implementation.

The papers would therefore be scientifically independent rather than two applications of an identical computational codebase.

A Six-Week Implementation Plan

This appendix records the intended implementation schedule. It is a project-management timeline rather than part of the computational specification proper, and is not binding on the scientific content defined in the preceding sections.

A.1 Week 1: Freeze scientific definitions

The first week is devoted primarily to eliminating ambiguity.

Prototype A

Finalize:

- MPS/MPO family;
- representability scale D;
- finite representation map;
- structural observable;
- structural complexity coordinate;
- reference calculation;
- primary parameter ranges.

Prototype B

Finalize:

- stabilizer/surface-code family;
- logical-operator definition;
- logical protection coordinate;
- finite logical subsystem;
- computational representation of $\Pi_D^{(\mathrm{code})}$;
- O_L evaluation strategy;
- code-size range.

Week 1 also finalizes the software stack (Section 1.1.12): the concrete choice of tensor-network library, quantum-circuit simulator, stabilizer simulator, and C++/Python binding technology must be fixed before Week 2 implementation begins. The same freeze records which independent reference implementations (Section 3.12.4) are used for the Level 2 small-system validation, and whether the optional GPU path (Section 1.1.10) is to be pursued at all.

At the end of Week 1, both the mathematical definitions and the software stack must be frozen.

A.2 Week 2: Minimal computational cores

Implement the smallest correct version of both prototypes.

For TN:

$$\text{state} \to \text{representation} \to \text{finite representation} \to F_{\mathrm{TN}}.$$

For QEC:

$$\text{code} \to \text{logical operators} \to \text{finite subsystem} \to O_L.$$

The priority is correctness, not performance.

A.3 Week 3: Validation infrastructure

Implement:

- unit tests;
- reference tests;
- convergence tests;
- reproducibility controls;
- experiment configuration;
- result serialization;
- baseline figures.

Only after these pass should performance optimization begin.

A.4 Week 4: Scaling experiments

Run systematic parameter sweeps.

Prototype A:

$$(D, N, d_{\mathrm{TN}}) \longrightarrow F_{\mathrm{TN}}.$$

Prototype B:

$$(D, d_L, N) \longrightarrow O_L.$$

Begin extracting scaling laws and obstruction boundaries.

A.5 Week 5: HPC optimization and final experiments

Move the dominant kernels to C++ where profiling justifies it.

Introduce:

- parallel execution;
- batched workloads;
- HPC benchmarking;
- optional hybrid quantum execution on small instances;
- optional GPU-accelerated contraction path for Prototype A (Section 1.1.10), only after the corresponding CPU results are validated.

Run the final statistically/numerically controlled experiments.

A.6 Week 6: Scientific consolidation

The final week is devoted to:

1. freezing datasets;
2. regenerating all flagship figures from clean runs;
3. performing all validation checks;
4. completing claim-traceability matrices;
5. documenting computational methodology;
6. benchmarking HPC performance;
7. preparing reproducibility artifacts;
8. drafting technical-paper-quality experimental narratives.

No major scientific feature should be introduced during the final week.